

Comparing LLM Architectures in Street Fighter III

by

Andrew Hinh
Hyemin Doo
Jack Wu

Submitted in partial fulfillment of the requirements
for the degree of
Bachelor of Science in Computer Science and Engineering
School of Engineering
Santa Clara University

Santa Clara, California
May 5, 2026

Comparing LLM Architectures in Street Fighter III

Andrew Hinh
Hyemin Doo
Jack Wu

Department of Computer Science and Engineering
Santa Clara University
May 5, 2026

ABSTRACT

We built a browser-based Street Fighter III benchmark for comparing large language models (LLMs) and vision-language models (VLMs) under real-time constraints. The game runs through a Gymnasium-like control loop that exposes both frames and structured state while the agent emits button actions. We deploy the system on Modal and stream matches over WebRTC, which lets us compare ability, latency, and cost in one setting instead of spreading them across unrelated benchmarks. Because the benchmark is narrow, architectural tradeoffs that are easy to blur in offline evaluation show up quickly during live play.

Table of Contents

1	Introduction	1
1.1	Problem Statement and Background	1
1.2	Objectives	2
1.3	Our Approach	2
2	User Research	3
2.1	Methods	3
2.2	Stakeholder Needs	3
2.3	User Stories	4
3	Design and Rationale	8
3.1	Architecture	8
3.2	End-to-End Runtime Operation	8
3.3	Use Case Descriptions	12
3.4	Use Case Diagram(s)	14
3.5	Class Diagram	15
3.6	Sequence Diagram	16
3.7	State Diagram	18
3.8	Component Diagram	19
3.9	Functional requirements	20
3.10	Non-functional requirements	20
4	Technologies	21
4.1	Backend and Deployment	21
4.2	Browser Client and Transport	21
4.3	Model Serving and Training	21
4.4	Evaluation and Testing	22
4.5	Developer Tooling	24
5	Implementation Plan	25
5.1	Timeline	25
5.2	Project Risks	27
6	Constraints and Standards	28
6.1	Constraints	28
6.2	Standards	29

7	System Evaluation	31
7.1	Testing Approach	31
7.2	Automated Test Coverage	31
7.3	Benchmark Results	32
7.4	Interpretation and Limits	33
8	Societal Issues	35
8.1	Ethical, Social, and Usability Considerations	35
8.2	Economic and Sustainability Considerations	36
9	Conclusion	37
	Acknowledgments	38
	Appendices	42
A.1	User Manual	42
A.1.1	End-User Prerequisites	42
A.1.2	Normal User Flow	42
A.2	Installation and Deployment Summary	43
A.3	Interface and API Summary	43
A.3.1	Browser-Facing Routes	43
A.3.2	WebRTC Data-Channel Messages	43
A.4	Selected Source Code Listings	44
A.4.1	Environment Adapter and Normalized Observation	44
A.4.2	Environment Reset, Character Selection, and Step Logic	45
A.4.3	Prompt Construction and Move Restriction	48
A.4.4	YOLO-Based Character Detection	51
A.4.5	Constrained Decoding for a Text-Only Model Server	53
A.4.6	Automated Evaluation Episode Loop	55
A.5	Latency and Throughput Calculations	58

List of Figures

2.1	Coin screen used to keep the first interaction fun.	5
2.2	Settings screen with participant selection, character choice, and match options before session start. . .	6
2.3	Controls-help overlay that keeps move and super-art reference inside the application.	7
3.1	High-level system architecture.	9
3.2	Runtime control loop from browser session startup through model move playback.	11
3.3	Use case summary for starting a match from the browser.	12
3.4	Use case summary for character, super-art, and match-option configuration.	12
3.5	Use case summary for selecting participants for player-versus-model or model-versus-model play. . .	13
3.6	Use case summary for opening the contextual controls-help overlay.	13
3.7	Use case summary for recovering from signaling, transport, or backend failure.	13
3.8	Use case diagram for the implemented player flows.	14
3.9	Class diagram of the session and transport logic.	15
3.10	Sequence diagram for a game session.	17
3.11	State diagram for a game session.	18
3.12	Component diagram of the deployed WebRTC architecture.	19
4.1	Participant serving stack used by the human player, multimodal models, and the text-only Nemotron baseline.	23
4.2	Observation flow for text-only versus multimodal participants before move playback.	23
5.1	Project timeline from initial scoping through evaluation and report work.	26
7.1	Elo ranking from the latest saved automated evaluation run.	33
7.2	Pairwise win-rate matrix for the same evaluation run.	34
A.1	Illustrative latency budget for LLM inference.	58

Chapter 1

Introduction

1.1 Problem Statement and Background

Large language models (LLMs) are usually compared through offline benchmarks, leaderboards, or task-specific demos. That is useful, but rarely in isolation. There are many different factors one might want to optimize for: maximum ability, ability per token, ability per dollar, ability per unit time, latency, and throughput. To make matters worse, as Raschka notes, “Comparing LLMs to determine the key ingredients that contribute to their good (or not-so-good) performance is notoriously challenging: datasets, training techniques, and hyperparameters vary widely and are often not well documented” [22].

Street Fighter III gives us a narrow but useful counterexample. The runtime exposes both rendered frames and structured metadata through a Gymnasium-like control loop, and the browser wrapper allows us to test ability, latency, and cost all at the same time. If a model is slow, brittle, or expensive, the setting makes that obvious quickly.

Recent architecture surveys help explain why this setting is useful. Raschka’s comparisons of open-weight model families argue that many practical differences now come from inference-oriented choices such as attention variants, MoE routing, KV-cache reduction, and related efficiency work rather than from abandoning the decoder-style baseline [22, 23]. As for multimodal models, Raschka’s overview of multimodal LLMs highlights two broad design patterns: systems that encode visual inputs before projecting them into the language model token space, and systems that rely more directly on cross-attention or unified token processing across modalities [21].

To capture a useful spread of models while staying within the latency limits discussed in Appendix A.5, we select recent open-weight models that can run on a single Nvidia B200 GPU. The multimodal roster includes Qwen 3.5 [19], Qwen 3.6 [20], Gemma 4 [5], and Ministral 3 [12]. Nemotron 3 Nano is the text-only baseline [15].

1.2 Objectives

- The system must support browser-based Street Fighter III matches in real time, including live player-versus-model sessions and fully automated model-versus-model sessions.
- The comparison should hold the game environment, frontend, transport stack, and deployment strategy constant so that the model choice is the main variable.
- The implementation should provide a practical base for later work on purpose-built model training, fine-tuning, and multimodal extensions.

1.3 Our Approach

We treat the application itself as the benchmark harness. The browser first loads a static Modal-hosted FastAPI app, then connects to a separate gameplay host for signaling and media. A WebSocket is used only for signaling, after which a browser-to-server WebRTC session carries video and a data channel for gameplay state and control messages. On the backend, each user session owns one local Street Fighter III runtime, while shared model services handle LLM or VLM inference. When a model is text-only, optional YOLO-based computer vision extracts coarse spatial information from frames before prompting the model; when a model is multimodal, JPEG-compressed frames are passed into the prompt directly. In the interface, player 1 may be human or model-controlled, while player 2 is always model-controlled.

Chapter 2

User Research

2.1 Methods

Our user-research process was lightweight and iterative rather than a formal human-subjects study. We began with a minimal playable prototype and then used internal play sessions and informal external playtesting to identify which parts of the experience mattered most. We watched where users hesitated, what required explanation, and which failures appeared. Findings from those sessions were then converted into implementation tasks and, when possible, automated tests.

This process changed the scope of the project. Features that sounded attractive at first, such as spectators, global leaderboards, persistent accounts, and two-human local play, mattered less than reliable startup, clear mode selection, responsive control, and graceful recovery when the connection failed. As a result, the shipped system prioritizes the gameplay loop itself: character and option selection, real-time frame delivery, two benchmark modes (player-versus-model and model-versus-model), contextual move help, and clear feedback when browser or backend conditions are not suitable for play.

2.2 Stakeholder Needs

- Players need a fast path from page load to a playable match. The first interaction should feel closer to launching a game than configuring a machine-learning demo.
- Players need controls that are legible before the match starts. Character choice, super-art selection, difficulty, and game mode must be understandable without reading source code or internal documentation.
- Players need low-latency visual feedback and robust failure handling. If the WebRTC session drops, if a backend service fails, or if the device is unsuitable for play, the system should say so clearly instead of leaving the user in an ambiguous state.

- Technical evaluators need a benchmark setting that exposes latency and gameplay quality under the same frontend and deployment stack. The main question is not only whether a model wins, but whether it remains usable during a live match.
- Developers and maintainers need a modular implementation that can be tested and modified independently. Frontend logic, session state, game-engine orchestration, and model-serving paths should be separable enough that regressions can be localized quickly.

2.3 User Stories

- As a player, I want to reach the settings screen and start a match quickly, so the system feels like a playable game and not a fragile demo.
- As a player, I want to choose characters, super arts, difficulty, and mode before a match, so I can shape the interaction I want to test.
- As a player, I want clear participant labels such as “YOU,” “QWEN3.5-35B,” and “NEMOTRON3-NANO-30B,” so I know who controls each side at all times.
- As a player, I want in-app controls help plus combo and super-art reference for the selected character, so I can play without memorizing Street Fighter III inputs in advance.
- As a mobile user, I want the application to warn me when I need landscape orientation or an external controller, so I do not enter a broken play state by accident.
- As a player, I want explicit recovery paths when the connection or backend fails, so I can return to the settings screen instead of refreshing the entire page.
- As a researcher or developer, I want the same environment to support both live player-versus-model sessions and fully automated model-versus-model sessions, so I can compare model behavior under a consistent application shell.

The shipped UI reflects those priorities directly. Figures 2.1 through 2.3 show the states that mattered most during playtesting: a fast startup screen, a dense but still legible settings screen, and an in-context help surface instead of an external manual.



Figure 2.1: Coin screen used to keep the first interaction fun.

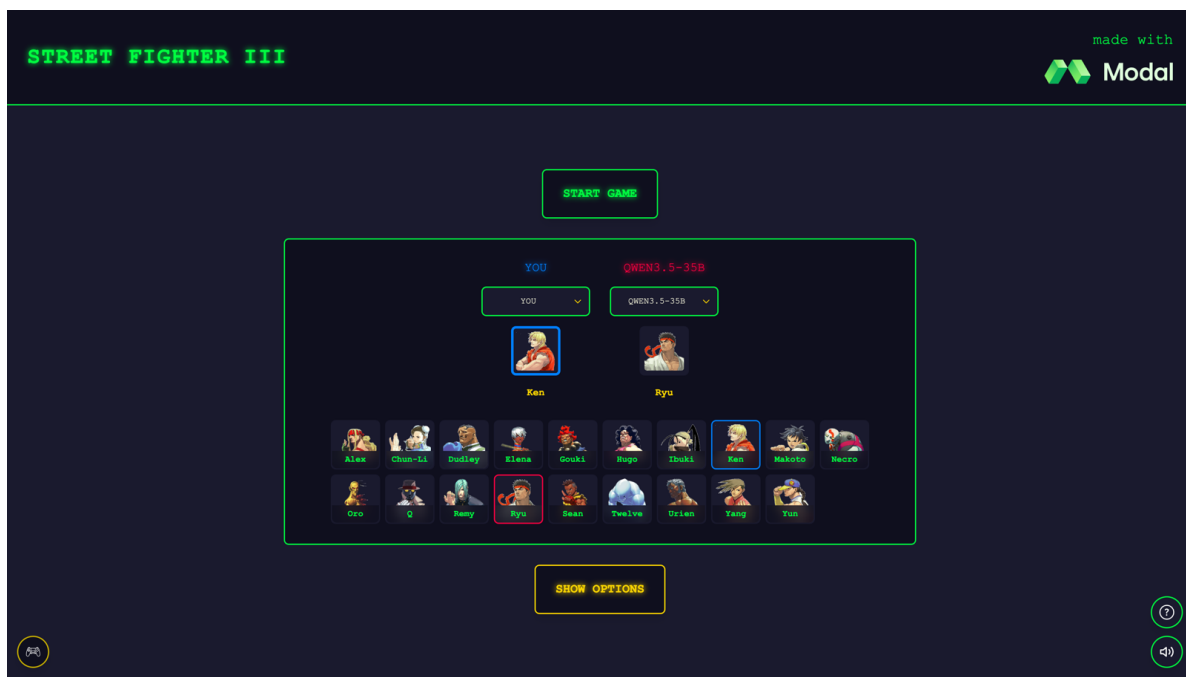


Figure 2.2: Settings screen with participant selection, character choice, and match options before session start.

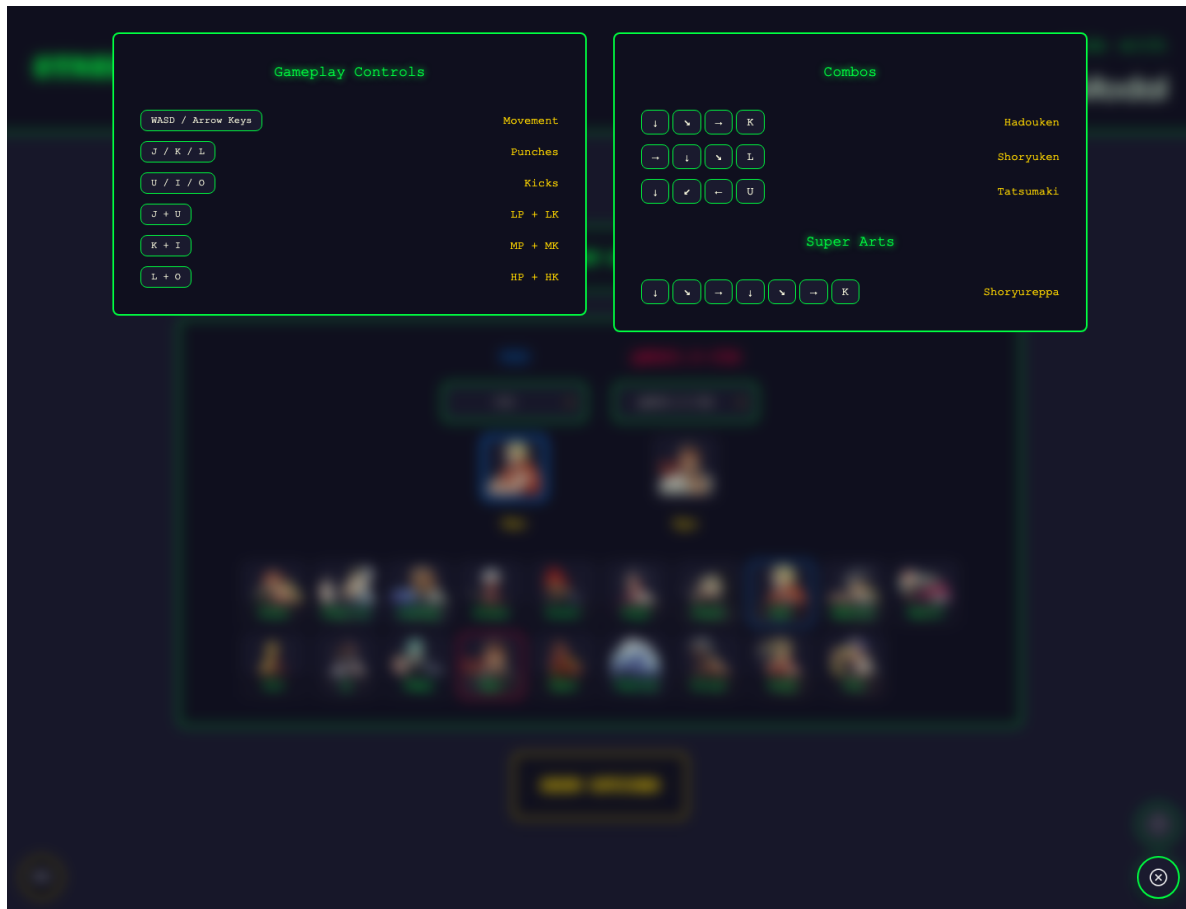


Figure 2.3: Controls-help overlay that keeps move and super-art reference inside the application.

Chapter 3

Design and Rationale

3.1 Architecture

Figure 3.1 shows the implemented system as three components: a browser layer for user interaction, input, and video display; a gameplay service for WebSocket signaling, WebRTC media, session state, and the game loop; and a model layer for the inference and perception services that turn emulator observations into Street Fighter button sequences. The split between the static assets and gameplay service keeps bursty asset requests separate from the heavyweight game loop. Altogether, this architecture supports components to utilize different hardware and scale separately.

3.2 End-to-End Runtime Operation

When a player opens the application, the browser first downloads the static frontend and runtime configuration from the static app. The client then opens a WebSocket to the gameplay app, requests TURN or STUN configuration, and negotiates a WebRTC peer connection. The WebSocket only carries signaling messages; once negotiation finishes, the remote video track carries emulator frames and the `game_control` data channel carries gameplay envelopes.

The `start_game` envelope contains participant choices, characters, outfits, super arts, difficulty, and gamepad status. On receipt, the gameplay service creates or reuses the local Street Fighter III runtime, prefetches required model servers, and resets the emulator into the requested match. The main loop then steps the emulator at the live-play cadence, updates the WebRTC video track with the latest frame, and sends status updates such as initializing, warming, running, finished, or error.

Model-controlled players run on a separate asynchronous loop so model latency does not directly stop frame delivery. Each decision uses the latest structured observation and either YOLO detections or JPEG-compressed frame data to build a prompt. The selected model returns a move name from the allowed move set; postprocessing resolves that name to button identifiers, and the session queues those identifiers. The game loop pops one queued action per

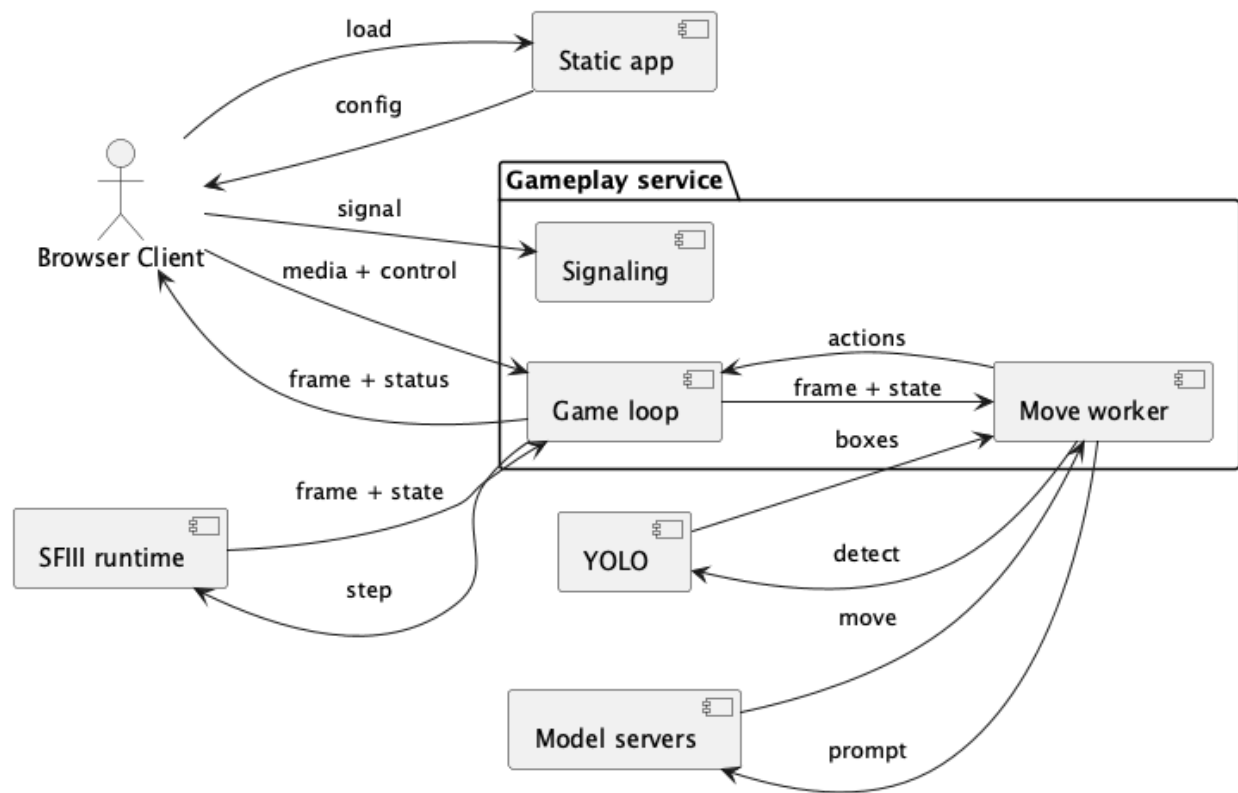


Figure 3.1: High-level system architecture.

frame and sends the corresponding MAME action values into the emulator.

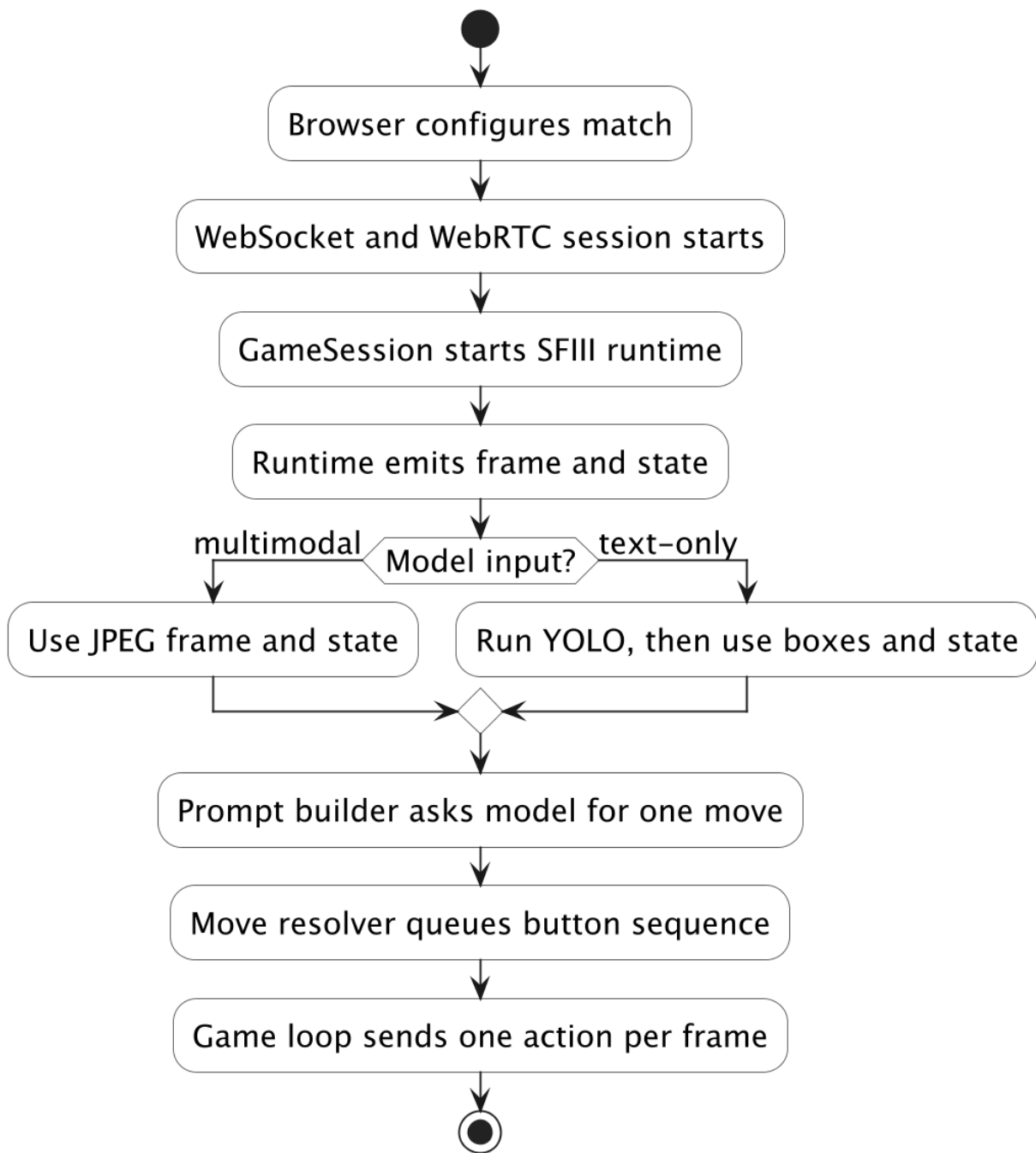


Figure 3.2: Runtime control loop from browser session startup through model move playback.

3.3 Use Case Descriptions

The use cases in this chapter come directly from the implemented system. They focus on startup, configuration, participant selection, move help, and failure recovery.

Use Case 1: Start a Match from the Browser
Actor: Player Preconditions: The application has loaded in a supported browser. Main Flow: 1. Player inserts a virtual coin and advances to the settings screen. 2. Player clicks “Start Game.” 3. System opens a live game session. 4. Player reaches the game screen. Alternate Flow: If session startup fails, the UI shows an explicit connection error. Postconditions: A playable match is running or the user receives explicit failure feedback.

Figure 3.3: Use case summary for starting a match from the browser.

Use Case 2: Configure Characters and Match Options
Actor: Player Preconditions: Player is on the settings screen. Main Flow: 1. Player opens the options panel. 2. Player chooses difficulty and super arts for each side. 3. Player selects characters and outfits. 4. System starts the next match with those settings. Alternate Flow: If a setting is unsupported for the current mode, the UI keeps the previous valid state. Postconditions: The next match starts with the chosen configuration.

Figure 3.4: Use case summary for character, super-art, and match-option configuration.

Use Case 3: Choose Participants and Match Mode

Actor: Player **Preconditions:** Player is on the settings screen.

Main Flow:

1. Player chooses whether player 1 is human-controlled or model-controlled.
2. Player chooses the model for player 2, and if needed, player 1.
3. System updates the labels for each side.
4. Player confirms the lineup before starting the match.

Alternate Flow: If a participant value is invalid, the system resets it to a valid default. **Postconditions:** The system and user agree on whether the session is player-versus-model or model-versus-model before the match begins.

Figure 3.5: Use case summary for selecting participants for player-versus-model or model-versus-model play.

Use Case 4: Open Controls Help and Move Lists

Actor: Player **Preconditions:** Player is on the settings screen.

Main Flow:

1. Player opens the help overlay.
2. System shows move and super-art information for the selected character.
3. Player reviews the controls.
4. Player closes the overlay and returns to settings.

Alternate Flow: If move data is unavailable, the overlay still opens with partial or empty content. **Postconditions:** Player receives contextual control assistance without leaving the application.

Figure 3.6: Use case summary for opening the contextual controls-help overlay.

Use Case 5: Recover from Connection or Backend Failure

Actor: Player **Preconditions:** Player has attempted to connect or is already in a match.

Main Flow:

1. A connection or backend error occurs.
2. System shows an explicit error state.
3. Player returns to the settings screen.
4. Player retries or changes settings.

Alternate Flow: If the session recovers before the player leaves, the match may continue. **Postconditions:** Player is not trapped in an undefined state after failure.

Figure 3.7: Use case summary for recovering from signaling, transport, or backend failure.

3.4 Use Case Diagram(s)

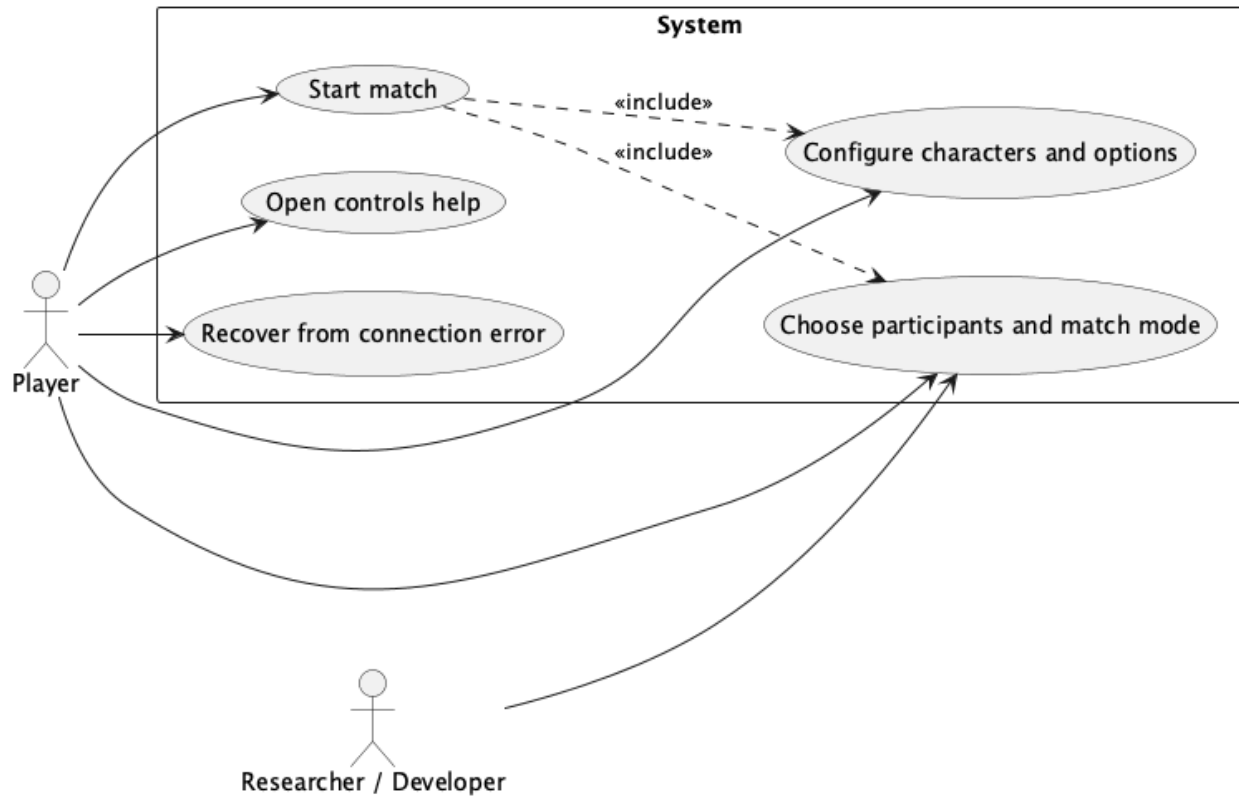


Figure 3.8: Use case diagram for the implemented player flows.

Figure 3.8 shows how the system starts matches, configures characters and super arts, chooses participants for player-versus-model or model-versus-model play, exposes move help, and recovers from failure.

3.5 Class Diagram

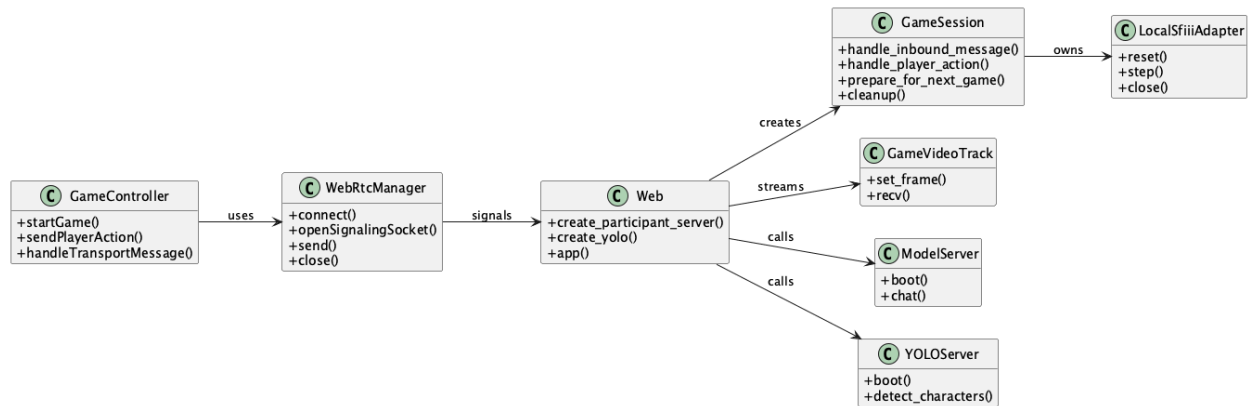


Figure 3.9: Class diagram of the session and transport logic.

The class view is centered on session ownership and transport boundaries. **Web** owns the Modal gameplay surface plus shared model-server handles, **GameSession** owns match-local state and button queues, and **GameVideoTrack** is only a transport helper that turns the latest emulator frame into a paced WebRTC stream. This separation matters because model latency should not directly stall frame delivery.

3.6 Sequence Diagram

The sequence diagram shows the two inference paths used in practice. Text-only models receive structured state plus YOLO detections, while multimodal models receive compressed frame data directly. In both cases, the selected move is converted into queued button presses, so inference runs in the background while the game loop continues stepping at the target cadence.

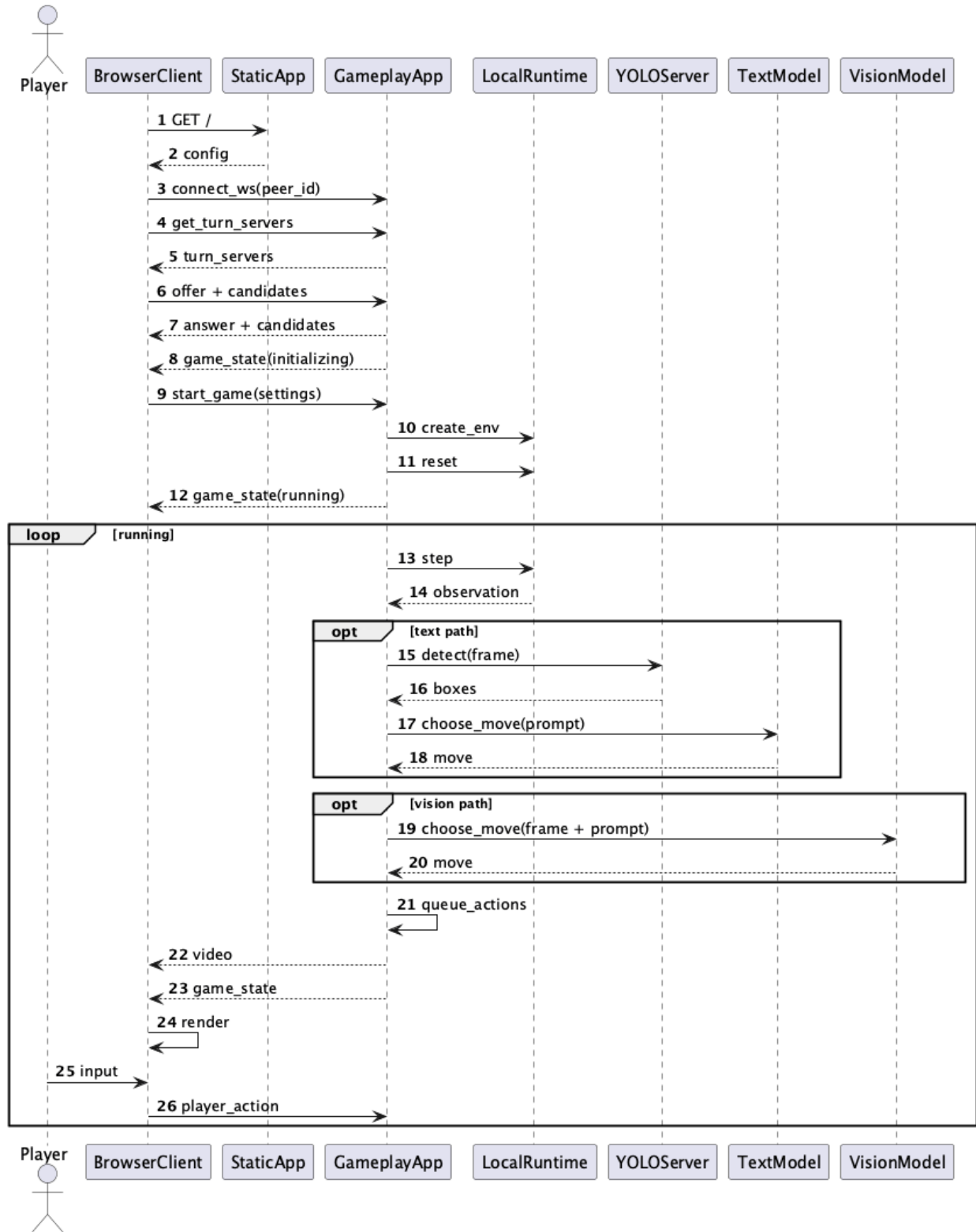


Figure 3.10: Sequence diagram for a game session.

3.7 State Diagram

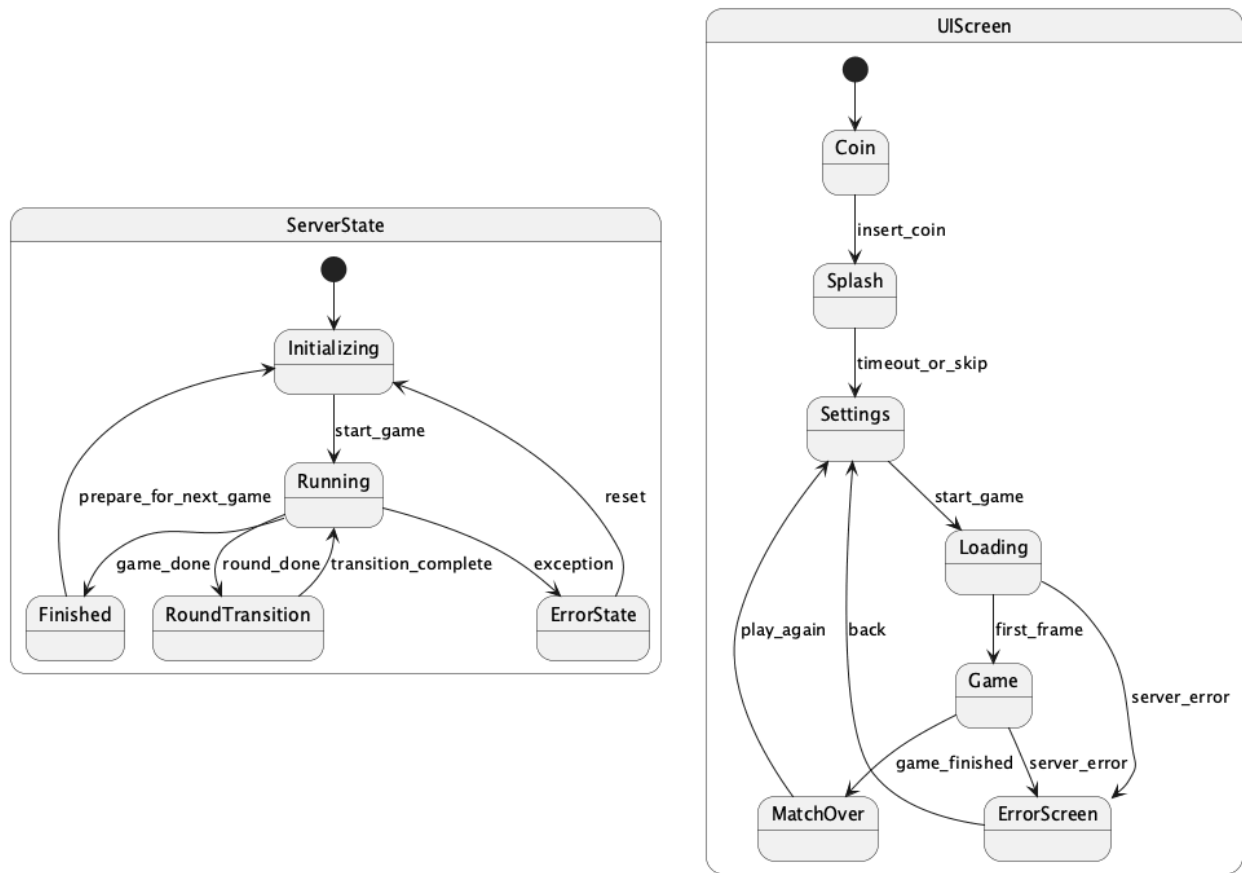


Figure 3.11: State diagram for a game session.

The state diagram is deliberately split into server and UI regions because most failures are cross-layer failures. A browser can still be on a loading or error screen while the backend is cleaning up a finished or failed session. Modeling both regions separately made it easier to reason about recovery paths and automated tests.

3.8 Component Diagram

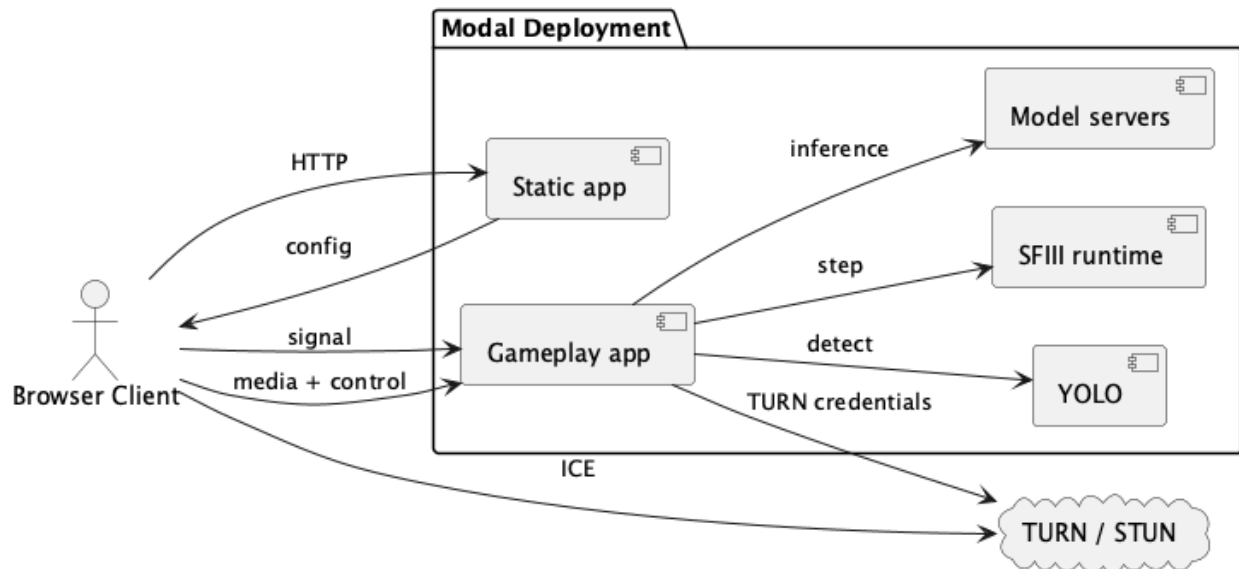


Figure 3.12: Component diagram of the deployed WebRTC architecture.

At the deployment level, the key architectural choice is the split between the static sf3 surface and the gameplay surface. Static assets can scale for bursty page loads without carrying the heavyweight gameplay dependencies, while gameplay containers keep the MAME runtime, WebRTC stack, and model prefetch logic colocated for lower end-to-end latency.

3.9 Functional requirements

1. The system must establish a browser-to-server game session that reaches a playable game screen from the browser, including WebSocket signaling, WebRTC negotiation, server-side session creation, and frame delivery.
2. The system must support the two implemented control configurations: player 1 versus an LLM or VLM, and LLM/VLM versus LLM/VLM. The interface must make it clear which side is controlled by a human and which side is controlled by a model.
3. The system must allow players to configure match settings before the game starts, including character selection, outfit, super art, and difficulty, and those settings must be sent to the backend as part of the start-game payload.
4. The system must provide contextual assistance and recovery features, including move-list help, mobile-device requirement gating, and explicit error handling when the signaling path, media path, or backend state fails.

3.10 Non-functional requirements

1. The system shall remain low-latency enough to preserve the feel of live play. This requirement motivates the use of browser-to-server WebRTC for media, colocated backend services where possible, and lightweight session state.
2. The system shall run in standard browsers without extra plugins. The interface shall support keyboard and gamepad input, adapt to different screen sizes, and block or warn against obviously unsuitable mobile conditions such as portrait orientation or missing controllers.
3. The system shall be reliable under partial failure. If TURN resolution falls back to STUN, if the WebRTC channel closes, or if the backend reports an error state, the user shall receive clear feedback instead of silent failure.
4. The system shall be maintainable and testable. The frontend, session state, environment lifecycle, and model services shall remain modular enough that backend unit tests, frontend unit tests, and end-to-end tests can catch regressions before deployment.

Chapter 4

Technologies

4.1 Backend and Deployment

The backend is Python on Modal. One lightweight FastAPI app serves the frontend and runtime configuration. Another handles gameplay: signaling, WebRTC session management, emulator ownership, and match orchestration.

The gameplay containers use aiortc for browser-to-server WebRTC, av for video-frame conversion, PyTurboJPEG for fast JPEG encoding, and MAMEToolkit for the Street Fighter III runtime. The local adapter in `src/env.py` translates emulator state into the observation format used everywhere else and applies queued button actions from humans or models. Modal volumes hold model caches, evaluation artifacts, and detector weights. Secrets from `.env` supply TURN credentials and model-access tokens.

4.2 Browser Client and Transport

The frontend is TypeScript under `src/frontend`. It implements the coin-to-settings-to-game flow, character and super-art selection, participant selection, keyboard and gamepad input, move-list help, and error recovery. A small runtime-config layer derives or injects the gameplay base URL so the browser can load static assets from one Modal hostname and then connect to the gameplay hostname for signaling and media.

A WebSocket handles signaling and TURN-server discovery. After negotiation, WebRTC carries the remote video track plus a data channel for gameplay envelopes such as `start_game`, `player_action`, and `gamepad_status`.

4.3 Model Serving and Training

The deployed roster is Qwen 3.5 35B, Qwen 3.6 35B, Gemma 4 31B, Ministral 3 14B, and Nemotron 3 Nano 30B, plus a human-controlled player-1 option. Qwen and Nemotron run using SGLang while Gemma and Ministral are served using vLLM. Qwen, Gemma, and Ministral are the multimodal participants. Nemotron is the only text-only

participant, so it calls a separate YOLO service, exported to ONNX and executed through ONNX Runtime with TensorRT, to recover coarse character positions from frames before prompting.

Observation construction. Each emulator step returns a rendered 384×224 RGB frame plus normalized scalar state: timer, side, health, stun, super meter, super count, and round wins for each player. The gameplay service stores the latest observation in the active `GameSession`. For multimodal models, the current frame is JPEG-compressed and encoded as a data URL before being passed into the chat request. For the text-only baseline, the current frame is sent to the YOLO server, which returns character boxes that are converted into relative distance and positioning language inside the prompt.

Prompting and constrained output. The prompt builder in `src/utils.py` combines the current match context, player state, recent movement or damage deltas, and a restricted list of legal high-level move names. Difficulty settings and super-meter availability determine which combos and super arts appear in that list. The model server is asked to return only the exact move name. SGLang-backed servers additionally constrain decoding with a regular expression over the allowed names, which prevents most malformed outputs before postprocessing.

Move playback. The model output is not sent directly to the emulator. Postprocessing first resolves the text into a named move and maps that move to a sequence of integer button identifiers. The session appends those identifiers to a bounded per-player queue. The live game loop then removes at most one queued action per player on each 60 Hz frame and translates it into the corresponding MAME action values.

The repo also includes training code for later experiments. The Qwen path uses self-play data collection plus LLaMA-Factory and TRL-style optimization code. The YOLO path prepares data, trains the detector, and exports deployable ONNX weights.

4.4 Evaluation and Testing

The repo includes an automated evaluation script, `src/evaluate.py`, for pairwise comparison. It reuses the same game runtime, prompt-construction logic, participant servers, and postprocessing path as the interactive system, then writes artifacts such as Elo rankings and win-rate matrices to a Modal volume.

Backend unit tests cover deterministic prompt, move, and postprocessing logic. Backend integration tests cover `GameSession` lifecycle and inbound protocol handling directly. Frontend Vitest suites in `src/frontend/__tests__` cover client-side state and transport helpers. Playwright covers a smaller set of mock-backed browser flows: coin insert, participant and settings submission, help-overlay behavior, mobile gating, explicit error handling, and back-to-back match resets.

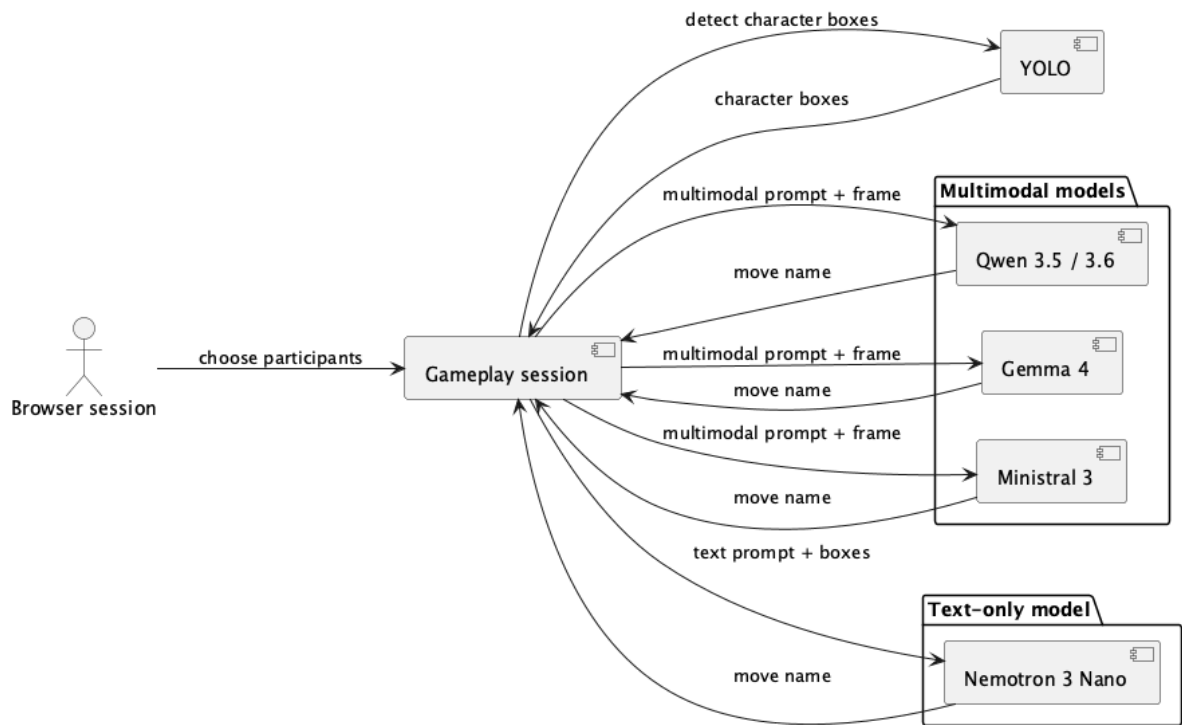


Figure 4.1: Participant serving stack used by the human player, multimodal models, and the text-only Nemotron baseline.

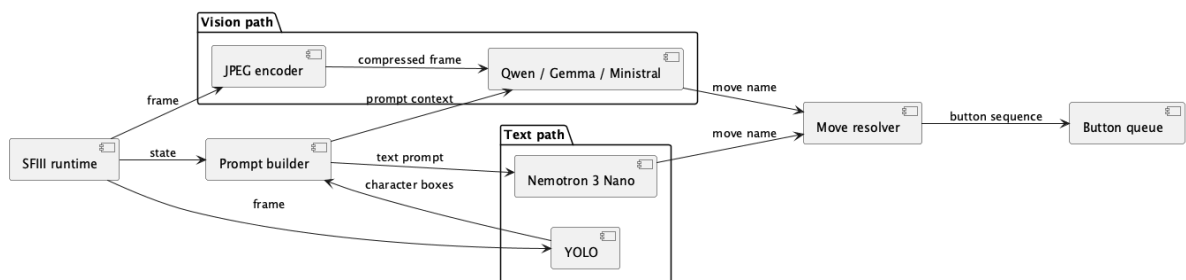


Figure 4.2: Observation flow for text-only versus multimodal participants before move playback.

4.5 Developer Tooling

The tooling follows the same split-language workflow. `uv` manages Python, `npm` manages the frontend, and `latexmk` builds the report. During local development, `mprocs` keeps the frontend watcher and `modal serve -m src.app` running together.

Chapter 5

Implementation Plan

5.1 Timeline

Figure 5.1 shows the implementation sequence. We first got the demo working, then cleaned it up and tested an initial reinforcement-learning path. In the new year we added more models and finished the WebRTC path. Around March, our original third-party game engine service stopped working, so we switched to a local emulator. We closed by adding model-selection controls and an evaluation script for pairwise testing.

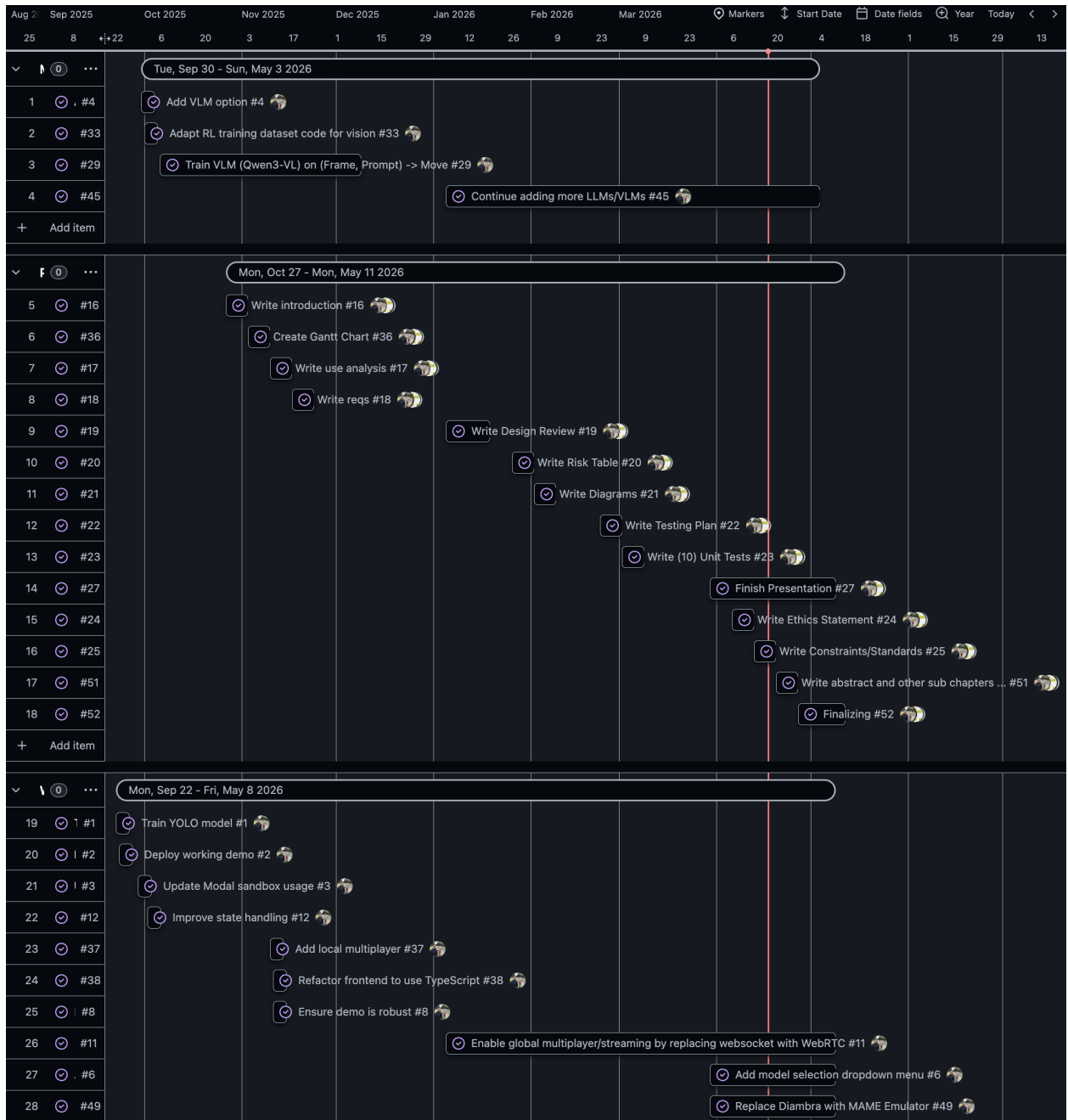


Figure 5.1: Project timeline from initial scoping through evaluation and report work.

5.2 Project Risks

1. Trained models may not outperform strong off-the-shelf baselines. This remained a real risk the whole time. Training is expensive, and the open-weight landscape changes quickly. Our mitigation was simple: make the platform useful even without custom checkpoints, and treat training as an extension rather than the core deliverable.

2. Real-time transport could add more complexity than value. Our initial websocket implementation mostly worked, and it was unclear if WebRTC would provide enough value to justify the complexity. We mitigated that by prototyping signaling and media early, then keeping the WebSocket only for signaling once the WebRTC path proved workable.

3. Weak test coverage could let regressions break core flows. The system crosses frontend, browser transport, emulator runtime, and model orchestration layers, so untested changes are unusually risky. We mitigated that by splitting testing across backend unit tests, frontend unit tests, and Playwright flows for the highest-value user paths.

4. Environment startup and cleanup could become too slow or unreliable. Each live match needs its own stateful game runtime. If runtime creation or cleanup were handled poorly, startup latency and cloud cost would rise quickly. We mitigated that by making session ownership explicit, prefetching environments when possible, and surfacing cleanup behavior in integration testing.

Chapter 6

Constraints and Standards

6.1 Constraints

1. **Budget and metered infrastructure.** The practical budget was about \$3000 all in Modal credits. That was enough for implementation, testing, and limited model experimentation, but not for unconstrained large-scale training or public production use. The main resources used were B200s for inference at \$6.25 per hour and 4x H200s for model training at \$18.16 per hour [13].
2. **End-to-end latency.** Street Fighter III is only a useful benchmark target if play feels live. Video delivery, control transport, emulator execution, perception, and inference all compete for the same narrow timing budget. That is why we rely on WebRTC for video delivery, datacenter-class GPUs for low-latency inference, and region colocation where possible.
3. **Licensed assets and controlled deployment.** The system relies on Street Fighter III assets and a properly managed ROM for the local emulator runtime. That makes the benchmark feasible as a university research prototype, but it also limits how broadly we can distribute it. Capcom’s fan-content guidance does not grant an unrestricted commercial license for derivative use [2].
4. **Reproducibility across hardware and hosted services.** Even with fixed application code, match behavior still depends on accelerator/region availability and relay conditions. The same dependencies that make the benchmark practical also make any one latency number or model ranking less portable across users and environments [13].

6.2 Standards

1. **WebRTC 1.0.** The main runtime path follows the WebRTC browser standard: the client creates a peer connection, negotiates a remote video track, and sends control traffic through a data channel [28].
2. **Interactive Connectivity Establishment, RFC 8445.** The WebRTC session is only usable if the two endpoints can discover a viable candidate pair across typical network boundaries. In practice, successful play depends on ICE candidate gathering and connectivity checks [7].
3. **Session Traversal Utilities for NAT, RFC 8489.** STUN provides the baseline mechanism for discovering public-facing transport addresses and keeping NAT bindings alive. There is also a fallback to a public STUN server when TURN credentials are missing or delayed [8].
4. **Traversal Using Relays around NAT, RFC 8656.** TURN is required when no direct path is available through ICE alone. TURN credentials are exposed as environment-backed secrets, and relay server configuration is returned to the browser during signaling [9].
5. **WebRTC Data Channels, RFC 8831.** After negotiation, gameplay control messages move on a dedicated data channel rather than the signaling WebSocket. RFC 8831 defines the low-overhead non-media transport we use inside the WebRTC session [10].
6. **WebSocket Protocol, RFC 6455.** The application uses a WebSocket for signaling. That path carries offer, answer, ICE-candidate, and TURN-server messages before gameplay transitions onto the WebRTC media and data-channel path [3].
7. **JSON, RFC 8259.** Signaling messages and gameplay envelopes use JSON serialization [1].
8. **HTTP Semantics, RFC 9110.** The system serves an HTML entry point, static images and audio assets, and JSON data for auxiliary gameplay information. Those browser-facing routes rely on ordinary HTTP request-response semantics even when the live match itself moves onto a real-time channel [11].
9. **HTML Living Standard.** The frontend depends on standard browser HTML behavior rather than a proprietary runtime. It uses ordinary buttons, forms, images, audio elements, canvas rendering, and DOM events, which helps portability across modern browsers and compatibility with accessibility tooling [25].
10. **Gamepad API.** The browser client listens for connection and disconnection events and polls controller state through the standard Gamepad API, so controller-assisted play does not require custom native software [27].

11. **ONNX Intermediate Representation.** The visual-perception path uses a YOLO model exported to ONNX and executed through ONNX Runtime. ONNX provides a standard model-exchange format, which keeps the perception component more portable than a framework-specific checkpoint would be [17].
12. **Web Content Accessibility Guidelines 2.2.** Even in a game benchmark, users need visible status, predictable controls, and understandable recovery paths. WCAG 2.2 is therefore relevant to start-button states, error messaging, screen transitions, and keyboard reachability [26].
13. **Unified Modeling Language 2.5.1.** The report uses UML-style diagrams to document system structure, use cases, and components [16].

Chapter 7

System Evaluation

Because the project spans a Python backend, a TypeScript frontend, browser media APIs, and remote model services, we evaluate it through several complementary layers rather than one monolithic test harness.

7.1 Testing Approach

Internal testing. We combine backend Pytest suites, frontend Vitest suites, Playwright browser tests, and direct manual playtesting whenever a change affects startup, controls, rendering, or mode selection.

External testing. External testing was lighter-weight and more formative than internal testing. We used informal play sessions with people outside the core implementation loop to check whether a new user could understand the coin-insert flow, the settings screen, the mode labels, and the mobile or controller requirements without additional explanation. Those sessions caught UX gaps quickly: if users cannot reach a live match, the benchmark is not testing the interaction we claim to evaluate.

7.2 Automated Test Coverage

Unit tests. Backend unit tests focus on deterministic helpers that feed model inference and session decisions. These tests cover move parsing and availability, super-bar counting, prompt and message construction, YOLO postprocessing, bounding-box handling, and fallback behavior when a model returns an invalid action.

On the frontend, the unit tests focus on UI state and client-side control logic. They cover runtime-config URL derivation, input mapping, combo and super-art recognition, asset loading, screen transitions, character-selection state, and WebRTC manager behavior such as disabling the start button on connection failure or buffering outbound messages until the data channel is open.

Integration tests. Backend integration tests exercise `GameSession` behavior directly. They verify that inbound

protocol messages mutate session state correctly, that human-input gating and action queueing behave as intended, that combos and super arts are queued properly, and that environment cleanup and next-game reset logic do not leave stale state behind.

In the browser, Playwright tests protect the main user flows through a mocked signaling and media backend rather than the full deployed stack. The critical suite covers the path from coin insert to the first rendered game frame, label updates after participant changes, and the settings flow that sends the correct start-game payload after character and difficulty changes. Additional end-to-end tests cover explicit error handling, mobile rotation and controller gating, help-overlay behavior, and back-to-back match handling.

7.3 Benchmark Results

Evaluations are performed using a direct model-comparison script in `src/evaluate.py`. It runs pairwise matches under the same Street Fighter III environment, prompt-construction logic, and model-serving stack used by the interactive application. The saved evaluation configuration fixes both sides to Ryu with the same outfit and super art, runs ten scheduled episodes for each unordered model pairing, and uses a faster `step_ratio=6` setting than the live browser path. In the latest saved run, five models were compared across ten pairings. One episode failed during environment creation, so the final dataset contained 99 completed matches.

A completed match is one automated episode that returns a winner label or a draw result before hitting the runtime guards. If environment creation or reset fails, the episode is skipped and excluded from the completed-match count. Draws are still completed matches: they represent episodes that finished normally but ended without either side having more round wins.

The Elo ranking uses the standard expected-score update implemented in `src/evaluate.py`. All models start at 1200, and each completed match updates the two participants with a K-factor of 16:

$$E_1 = \frac{1}{1 + 10^{(R_2 - R_1)/400}}, \quad R'_1 = R_1 + 16(S_1 - E_1).$$

The second player's expected score is computed symmetrically. A win gives the winner score $S = 1$ and the loser $S = 0$; a draw gives both models $S = 0.5$. The plotted Elo values therefore summarize the ordered match history, not a direct percentage of wins.

The win-rate matrix is constructed from unordered model pairings. For each pair, the row-model entry is its number of wins divided by completed episodes for that pair, and the mirrored cell stores the other model's win rate. Draws remain in the denominator but are not credited to either model, so two mirrored cells can sum to less than one when a matchup is draw-heavy.

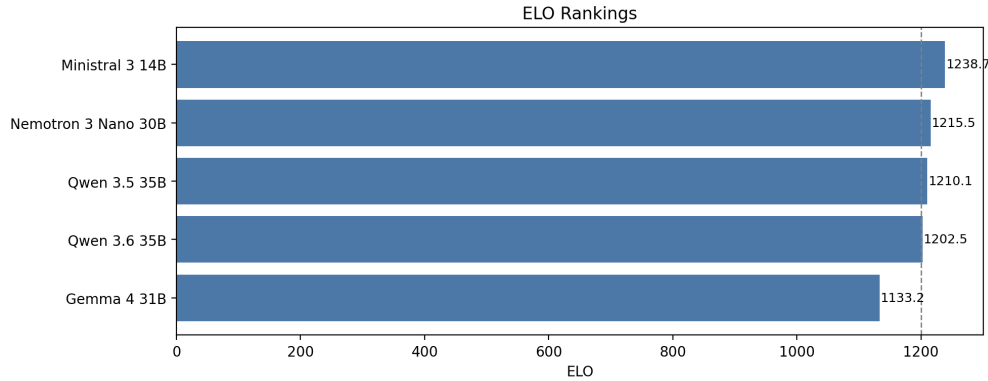


Figure 7.1: Elo ranking from the latest saved automated evaluation run.

Figure 7.1 shows the aggregate ranking from that run. Ministral 3 14B finished first with an Elo of 1238.69, followed by Nemotron 3 Nano 30B at 1215.55, Qwen 3.5 35B at 1210.12, Qwen 3.6 35B at 1202.46, and Gemma 4 31B at 1133.18.

Figure 7.2 shows how draw-heavy the benchmark remains. Several matchups ended mostly or entirely in draws, including Ministral versus Nemotron and Ministral versus Qwen 3.5. The result points to more than one-dimensional model strength: different policies also vary in how often they convert neutral play into decisive rounds under the current latency and prompting constraints.

7.4 Interpretation and Limits

These benchmark results should be interpreted cautiously. First, the sample size is still small enough that one or two decisive pairings can move the ranking. Second, the evaluation reuses the same infrastructure stack as the live application, so environment startup, model warmup, and cloud variability can affect outcomes. The latest run itself surfaced this: one Ministral 3 14B versus Qwen 3.5 35B episode was skipped because environment creation timed out.

The saved run is also narrower than the full interactive system. It does not vary character choice, super-art choice, or live human input, and its accelerated stepping configuration is better suited to offline comparison than to user-facing play. Even with those caveats, the results show that the platform can run repeatable pairwise comparisons and reveal both decisive wins and draw-heavy failure modes.

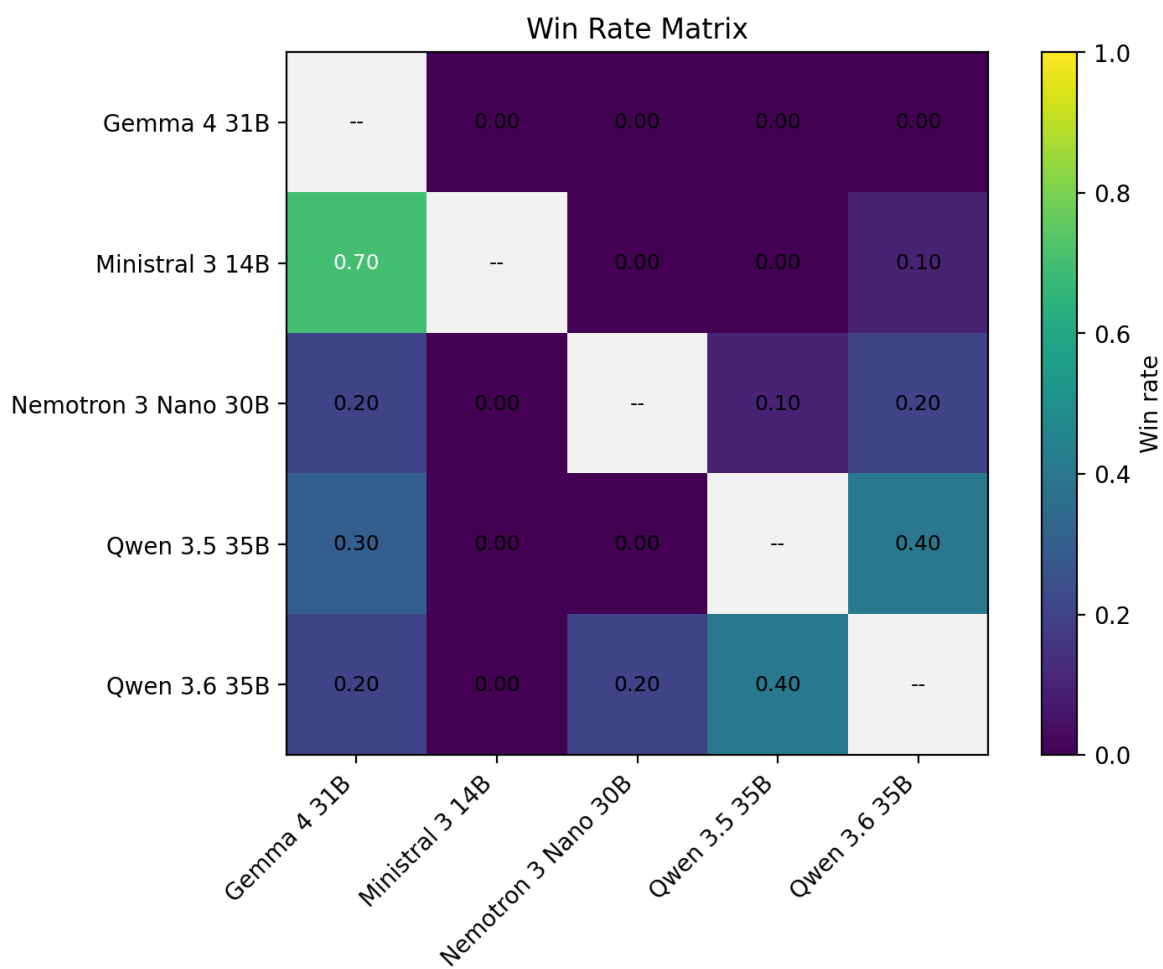


Figure 7.2: Pairwise win-rate matrix for the same evaluation run.

Chapter 8

Societal Issues

8.1 Ethical, Social, and Usability Considerations

Ethical issues. Even though this project is mostly a game demo, it is still a public-facing AI system whose internals are not obvious to users. That means we have to be clear about what it does and does not show. Benchmark quality here depends on latency, model choice, external services, and the narrow game setting, not on any broad claim of intelligence. The use of licensed game assets also creates straightforward obligations around intellectual property and responsible deployment. More broadly, the system should be judged by whether it benefits users and makes responsibility for design choices legible, which aligns with both the ACM/IEEE software engineering code and leading AI ethics frameworks [6, 4, 14].

Social access. The project moves human-LLM interaction away from passive text chat and toward something more observable and interactive. Play surfaces latency, ability, and failure immediately. At the same time, the project is not equally accessible to everyone. Its quality depends on a stable network connection, a capable device, familiarity with fighting games, and in some cases a physical controller, so it is more welcoming to technically equipped players than to casual or resource-constrained users. That access gap should stay visible when interpreting user-facing results [4, 14].

Usability. The headline claim is real-time browser play against LLMs, so controls, error messages, and device gating matter as much as model accuracy. The design supports standard browsers, keyboard input, gamepad input, feedback on connection failures, and mobile gating that warns users when landscape orientation or an external input device is required. However, the high-speed gameplay, rapid state changes, and controller assumptions can still exclude users with motor, visual, or cognitive constraints, so inclusive design remains important. Standards such as WCAG 2.2 matter here because keyboard access, visible focus, and predictable interaction remain essential even in a game-oriented interface [26].

8.2 Economic and Sustainability Considerations

Economics. Cloud-hosted inference adds recurring cost beyond ordinary web hosting. Training or serving YOLO, VLM, and LLM models under low-latency constraints gets expensive quickly, so model quality has to be weighed against throughput, concurrency, and budget. Deployment also requires valid credentials, legally obtained game assets, and enough compute to run the system at all [13]. For that reason, the project makes more sense as a research prototype than as a mass-market product.

Lifecycle sustainability. In the narrower engineering sense, sustainability means whether the system can remain useful and maintainable over time. Long-term viability is still constrained by dependence on Modal deployment, TURN credentials, and access to specific game assets. If any of those dependencies change materially, the system becomes harder to reproduce or maintain. Sustainable operation depends not only on code quality, but also on documenting interfaces, limiting unnecessary platform lock-in, and managing AI-specific risks across the lifecycle [14].

Environmental impact. Both model training and repeated inference consume nontrivial compute resources: reinforcement learning and self-play can require many environment steps, while real-time gameplay encourages frequent inference and low-latency serving. Prior work shows that the environmental cost of machine learning depends heavily on model efficiency, hardware choice, cloud region, and how often training is repeated [24, 18]. For this project, environmental responsibility means keeping the benchmark narrow, preferring efficient architectures when possible, and avoiding unnecessary retraining [4, 14].

Chapter 9

Conclusion

We ended with a working real-time benchmark for comparing language-model architectures inside a live browser game instead of only through offline evaluation. The final system combines a browser frontend, browser-to-server WebRTC, split static and gameplay FastAPI surfaces on Modal, a local MAME-based game runtime, and an optional text-only perception path. In practice, that gives us the two configurations that mattered most to the thesis question: live player-versus-model sessions and fully automated model-versus-model sessions.

Once a model has to play a live match, latency and recovery can be experienced interactively, making it much harder to hide weaknesses behind aggregate benchmark scores. It also reinforces the less convenient point from recent architecture surveys: even in a controlled application, model comparison is still shaped by hardware, inference stacks, warmup effects, and training history.

The platform is modular, testable, and close to the user experience it studies. It also supports automated pairwise evaluation inside the same application shell; in the latest saved run, Ministral 3 14B ranked first and Gemma 4 31B ranked last, with several draw-heavy matchups in between. It also leaves room for later custom training work.

However, the deployment depends on external infrastructure and the game assets are licensed, which rules out unrestricted distribution. The platform therefore fits better as a research and demonstration system than as a commercial game service. Natural next steps include comparing more LLMs and VLMs under the same shell and testing training paradigms such as online reinforcement learning.

Acknowledgments

We are grateful to Modal for financially supporting this project. Modal provided a monthly spend credit of \$500 and a one-time credit of \$2,500, which made the web deployment and model experiments possible. We also thank Alessandro Palmas for his help with earlier game-environment debugging and integration work. Finally, we thank our advisors for guiding the design, implementation, and writing process from start to finish.

Bibliography

- [1] Tim Bray. The javascript object notation (JSON) data interchange format, 2017. URL <https://www.rfc-editor.org/rfc/rfc8259>. RFC 8259.
- [2] Capcom. Capcom fan content guidelines, 2025. URL <https://www.capcom-games.com/en/fan-content-guidelines/>. Accessed April 7, 2026.
- [3] Ian Fette and Alexey Melnikov. The WebSocket protocol, 2011. URL <https://www.rfc-editor.org/rfc/rfc6455>. RFC 6455.
- [4] Luciano Floridi and Josh Cowls. A unified framework of five principles for AI in society. *Harvard Data Science Review*, 2019. URL <https://hdsr.mitpress.mit.edu/pub/10jsh9d1>. Accessed April 7, 2026.
- [5] Google. Gemma 4: Our most capable open models to date, 2026. URL <https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/>. Accessed April 20, 2026.
- [6] Don Gotterbarn, Keith Miller, and Simon Rogerson. Software engineering code of ethics and professional practice, 1999. URL <https://www.acm.org/code-of-ethics/software-engineering-code>. Accessed April 7, 2026.
- [7] Internet Engineering Task Force. Interactive connectivity establishment (ICE): A protocol for network address translator (NAT) traversal, 2018. URL <https://www.rfc-editor.org/rfc/rfc8445>. RFC 8445.
- [8] Internet Engineering Task Force. Session traversal utilities for NAT (STUN), 2020. URL <https://www.rfc-editor.org/rfc/rfc8489>. RFC 8489.
- [9] Internet Engineering Task Force. Traversal using relays around NAT (TURN): Relay extensions to session traversal utilities for NAT (STUN), 2020. URL <https://www.rfc-editor.org/rfc/rfc8656>. RFC 8656.
- [10] Internet Engineering Task Force. WebRTC data channels, 2021. URL <https://www.rfc-editor.org/rfc/rfc8831>. RFC 8831.

- [11] Internet Engineering Task Force. Http semantics, 2022. URL <https://www.rfc-editor.org/rfc/rfc9110>. RFC 9110.
- [12] Alexander H. Liu et al. Ministral 3, 2026. URL <https://arxiv.org/abs/2601.08584>. arXiv:2601.08584, accessed April 21, 2026.
- [13] Modal Labs. Modal pricing, 2026. URL <https://modal.com/pricing>. Accessed April 7, 2026.
- [14] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). Technical report, National Institute of Standards and Technology, 2023. URL <https://doi.org/10.6028/NIST.AI.100-1>. Accessed April 7, 2026.
- [15] NVIDIA. Nemotron 3 nano: Open, efficient mixture-of-experts hybrid mamba-transformer models. Technical report, NVIDIA, 2026. URL <https://research.nvidia.com/labs/nemotron/files/NVIDIA-Nemotron-3-Nano-Technical-Report.pdf>. Accessed April 20, 2026.
- [16] Object Management Group. Unified modeling language (UML) version 2.5.1, 2017. URL <https://www.omg.org/spec/UML/2.5.1/>. Accessed April 7, 2026.
- [17] ONNX Project. Open neural network exchange intermediate representation (ONNX IR) specification, 2026. URL <https://onnx.ai/onnx/repo-docs/IR.html>. Accessed April 14, 2026.
- [18] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. The carbon footprint of machine learning training will plateau, then shrink. *Computer*, 2022. URL <https://arxiv.org/abs/2204.05149>. Accessed April 7, 2026.
- [19] Qwen Team. Qwen3.5: Towards native multimodal agents, 2026. URL <https://qwen.ai/blog?id=qwen3.5>. Accessed April 21, 2026.
- [20] Qwen Team. Qwen3.6-35b-a3b: Agentic coding power, now open to all, 2026. URL <https://qwen.ai/blog?id=qwen3.6-35b-a3b>. Accessed April 20, 2026.
- [21] Sebastian Raschka. Understanding multimodal LLMs, 2024. URL <https://magazine.sebastianraschka.com/p/understanding-multimodal-llms>. Accessed April 7, 2026.
- [22] Sebastian Raschka. The big LLM architecture comparison, 2025. URL <https://magazine.sebastianraschka.com/p/the-big-llm-architecture-comparison>. Accessed April 7, 2026.

- [23] Sebastian Raschka. A dream of spring for open-weight models, 2026. URL <https://magazine.sebastianraschka.com/p/a-dream-of-spring-for-open-weight>. Accessed April 7, 2026.
- [24] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in NLP. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3645–3650, 2019. URL <https://arxiv.org/abs/1906.02243>.
- [25] WHATWG. Html standard, 2026. URL <https://html.spec.whatwg.org/multipage/>. Living Standard, accessed April 14, 2026.
- [26] World Wide Web Consortium. Web Content Accessibility Guidelines (WCAG) 2.2, 2023. URL <https://www.w3.org/TR/WCAG22/>. Accessed April 7, 2026.
- [27] World Wide Web Consortium. Gamepad, 2025. URL <https://www.w3.org/TR/gamepad/>. Working Draft, accessed April 14, 2026.
- [28] World Wide Web Consortium. WebRTC: Real-time communication in browsers, 2025. URL <https://www.w3.org/TR/webrtc/>. Accessed April 7, 2026.

Appendices

A.1 User Manual

This appendix is for a third party who needs to run or extend the system.

A.1.1 End-User Prerequisites

- A modern browser with WebRTC support.
- A stable network connection, since video is streamed from the backend in real time.
- For mobile play, landscape orientation plus an external keyboard or gamepad.
- For local development or deployment, a legal copy of the Street Fighter III ROM plus the required credentials described in the setup instructions below.

A.1.2 Normal User Flow

1. Open the deployed static application URL in a browser.
2. Wait for the splash and coin-insert screens, then advance to the settings screen.
3. Choose whether player 1 is controlled by a human or a model. Choose the model that controls player 2.
4. Optionally configure character, outfit, super art, and difficulty for both sides, and open the controls-help overlay to inspect combos and super-arts for the selected character.
5. Click `Start Game`. The browser opens a signaling WebSocket, resolves TURN or STUN information, negotiates a WebRTC session, and then transitions to the live game screen.
6. During play, keyboard or gamepad inputs are sent over the WebRTC data channel. If the session fails, the interface surfaces an explicit error state and provides a path back to settings.

A.2 Installation and Deployment Summary

The README contains the canonical commands. A minimal reproduction flow is:

```
uv sync --all-groups
source .venv/bin/activate
npm i
prek install
```

Then provide the Street Fighter III ROM at `assets/engine/sfiii3n.zip` and create a `.env` file with the required credentials:

```
TURN_USERNAME=
TURN_CREDENTIAL=
WANDB_API_KEY=
HF_TOKEN=
OPENAI_API_KEY=
```

For local development, the intended workflow is:

```
mprocs
```

which runs the frontend watcher together with `modal serve -m src.app`. For production deployment, the repository uses:

```
npm run build && modal deploy -m src.app
```

The automated evaluation harness is launched separately with:

```
modal run -m src.evaluate
```

A.3 Interface and API Summary

A.3.1 Browser-Facing Routes

- `/`: serves the static frontend endpoint.
- `/runtime-config.js`: injects the gameplay base URL into the browser.
- `/api/extra-moves`: returns combo and special-move metadata used by the help overlay.
- `/ws/{peer_id}`: signaling WebSocket used for offer, answer, ICE-candidate, and TURN-server messages before live gameplay begins.

A.3.2 WebRTC Data-Channel Messages

After signaling completes, small JSON envelopes move over the `game_control` data channel. The main inbound message types are:

- `start_game`: sends participant choices plus match settings such as character, outfit, super art, difficulty, and whether a gamepad is connected.
- `player_action`: sends low-level actions or named combo and super-art requests.
- `gamepad_status`: informs the backend whether a controller is attached.

The main outbound message type is `game_state`, which carries:

- `status`: one of initializing, warming, running, finished, or error,
- `scores`: round wins for each side,
- `winner`: winner label after a finished match,
- `error`: human-readable failure text when the session enters an error state.

A.4 Selected Source Code Listings

These listings cover the main environment, prompting, perception, decoding, and evaluation paths.

A.4.1 Environment Adapter and Normalized Observation

Listing A.1: Environment configuration and normalized observation schema from `src/env.py`.

```
@dataclass(frozen=True)
class EnvironmentConfig:
    characters: tuple[str, str]
    outfits: tuple[int, int]
    super_arts: tuple[int, int]
    step_ratio: int
    render_mode: str = "rgb_array"
    disable_keyboard: bool = False
    disable_joystick: bool = False
    roms_path: str = "/root"
    env_id: str | None = None

    def resolved_env_id(self) -> str:
        return self.env_id or next_env_id()

class GameEnvironment(Protocol):
    def reset(self) -> tuple[dict[str, Any], dict[str, Any]]: ...

    def step(
        self,
        actions: dict[str, int],
    ) -> tuple[dict[str, Any], float, bool, bool, dict[str, Any]]: ...

    def close(self) -> None: ...

SUPER_BAR_MAX = 128
```

```

STUN_BAR_MAX = 72
TIMER_MAX = 100

def _scalar(value: Any) -> int:
    if np is not None and isinstance(value, np.ndarray):
        return int(value.reshape(-1)[0])
    if isinstance(value, (list, tuple)):
        return int(value[0])
    return int(value)

def _array1(value: int, *, dtype: Any = None) -> Any:
    if np is None:
        raise ModuleNotFoundError("numpy is required for gameplay runtime")
    return np.array([value], dtype=dtype or np.int16)

def _normalize_player(
    raw: Mapping[str, Any],
    *,
    prefix: str,
    wins: int,
) -> dict[str, Any]:
    side = _scalar(raw[f"side{prefix}"])
    stun_timer = _scalar(raw[f"stunTimer{prefix}"])
    stun_bar = _scalar(raw[f"stunBar{prefix}"])
    super_bar = _scalar(raw[f"superGauge{prefix}"])
    super_count = _scalar(raw[f"superCount{prefix}"])
    health = _scalar(raw[f"health{prefix}"])

    return {
        "wins": _array1(wins),
        "side": side,
        "stunned": stun_timer > 0,
        "stun_bar": _array1(max(0, min(STUN_BAR_MAX, stun_bar))),
        "health": _array1(health),
        "super_count": _array1(max(0, super_count)),
        "super_bar": _array1(max(0, min(SUPER_BAR_MAX, super_bar))),
    }

def normalize_local_observation(raw: Mapping[str, Any]) -> dict[str, Any]:
    timer = max(0, min(TIMER_MAX, _scalar(raw["timer"])))
    wins_p1 = _scalar(raw["winsP1"])
    wins_p2 = _scalar(raw["winsP2"])

    return {
        "frame": raw["frame"],
        "timer": _array1(timer),
        "P1": _normalize_player(raw, prefix="P1", wins=wins_p1),
        "P2": _normalize_player(raw, prefix="P2", wins=wins_p2),
    }

```

A.4.2 Environment Reset, Character Selection, and Step Logic

Listing A.2: Local runtime reset, character-selection control, and reward-bearing step logic from `src/env.py`.

```

def reset(self) -> tuple[dict[str, Any], dict[str, Any]]:
    self._new_game()
    return normalize_local_observation(self._data), {}

def step(

```

```

self,
actions: dict[str, int],
) -> tuple[dict[str, Any], float, bool, bool, dict[str, Any]]:
    raw = self._sub_step(
        _action_values(1, actions.get("agent_0", 0))
        + _action_values(2, actions.get("agent_1", 0))
    )
    reward = float(raw["reward"])
    round_done = False
    game_done = False

    if int(raw["fighting"]) == 0:
        raw = self._run_till_victor(raw)
        reward = float(raw["reward"])
        round_done = True
        game_done = int(raw["winsP1"]) >= 2 or int(raw["winsP2"]) >= 2
        if not game_done:
            self._data = self._wait_for_fight_start()
            self._data["reward"] = reward
        else:
            self._data = raw
    else:
        self._data = raw

    observation = normalize_local_observation(self._data)
    info = {
        "game_done": game_done,
        "round_done": round_done,
        "stage_done": False,
    }
    return observation, reward, game_done, False, info

def close(self) -> None:
    self.emu.close()

def _run_steps(self, steps: list[dict[str, Any]]) -> None:
    for step in steps:
        for _ in range(step["wait"]):
            self._data = self.emu.step([])
        actions = [action.value for action in step["actions"]]
        self._data = self.emu.step(actions)

def _write_u8(self, address: int, value: int) -> None:
    self.emu.console.writeln(f"mem:write_u8({hex(address)}, {int(value)})")

def _wait_for_continue(self) -> None:
    stable_frames = 0
    for _ in range(240):
        self._data = self.emu.step([])
        if (
            int(self._data["characterSelectStateP1"]) == 0
            and int(self._data["characterSelectStateP2"]) == 0
        ):
            stable_frames += 1
            if stable_frames >= 5:
                return
        else:
            stable_frames = 0
    print("Warning: continue transition did not settle; proceeding anyway")

def _wait_for_character_select(self) -> None:
    from MAMEToolkit.sf_environment.Actions import Actions

    for _ in range(240):
        self._data = self.emu.step([])

```

```

        state_p1 = int(self._data["characterSelectStateP1"])
        state_p2 = int(self._data["characterSelectStateP2"])
        if state_p1 >= 2 and state_p2 >= 2:
            return
        if state_p1 == 0 or state_p2 == 0:
            self._data = self.emu.step(
                [Actions.P1_START.value, Actions.P2_START.value]
            )
        raise TimeoutError("Timed out waiting for 2-player character select")

def _select_characters(self) -> None:
    from MAMEToolkit.sf_environment.Actions import Actions

    p1_char = CHARACTER_NAME_TO_LOCAL_ID[self.config.characters[0]]
    p2_char = CHARACTER_NAME_TO_LOCAL_ID[self.config.characters[1]]
    p1_color = max(0, self.config.outfits[0] - 1)
    p2_color = max(0, self.config.outfits[1] - 1)
    p1_sa = max(0, self.config.super_arts[0] - 1)
    p2_sa = max(0, self.config.super_arts[1] - 1)

    character_locked = {"P1": False, "P2": False}
    sa_locked = {"P1": False, "P2": False}

    for _ in range(240):
        self._data = self.emu.step([])
        state_p1 = int(self._data["characterSelectStateP1"])
        state_p2 = int(self._data["characterSelectStateP2"])

        self._write_u8(0x02011387, p1_char)
        self._write_u8(0x02011388, p2_char)
        self._write_u8(0x02015683, p1_color)
        self._write_u8(0x02015684, p2_color)

        pressed = []
        if state_p1 == 2 and not character_locked["P1"]:
            pressed.append(Actions.P1_JPUNCH.value)
            character_locked["P1"] = True
        if state_p2 == 2 and not character_locked["P2"]:
            pressed.append(Actions.P2_JPUNCH.value)
            character_locked["P2"] = True

        if state_p1 >= 3:
            self._write_u8(0x020154D3, p1_sa)
        if state_p2 >= 3:
            self._write_u8(0x020154D5, p2_sa)

        if state_p1 == 4 and not sa_locked["P1"]:
            pressed.append(Actions.P1_JPUNCH.value)
            sa_locked["P1"] = True
        if state_p2 == 4 and not sa_locked["P2"]:
            pressed.append(Actions.P2_JPUNCH.value)
            sa_locked["P2"] = True

        if pressed:
            self._data = self.emu.step(pressed)
            state_p1 = int(self._data["characterSelectStateP1"])
            state_p2 = int(self._data["characterSelectStateP2"])

        if state_p1 == 5 and state_p2 == 5:
            return

    raise TimeoutError("Timed out locking characters/super arts")

def _wait_for_fight_start(self) -> dict[str, Any]:
    from MAMEToolkit.sf_environment.Actions import Actions

```

```

self._data = self.emu.step([])
for frame_idx in range(900):
    if int(self._data["fighting"]) != 0:
        break
    if frame_idx > 0 and frame_idx % 180 == 0:
        # Skip long intros / continue prompts if they linger.
        self._data = self.emu.step(
            [Actions.P1_START.value, Actions.P2_START.value]
        )
    else:
        self._data = self.emu.step([])
else:
    raise TimeoutError(
        "Timed out waiting for fight start "
        f"(p1_state={int(self._data['characterSelectStateP1'])}, "
        f"p2_state={int(self._data['characterSelectStateP2'])}, "
        f"fighting={int(self._data['fighting'])})"
    )

self.expected_health = {
    "P1": int(self._data["healthP1"]),
    "P2": int(self._data["healthP2"]),
}
self.expected_wins = {
    "P1": int(self._data["winsP1"]),
    "P2": int(self._data["winsP2"]),
}
return self._sub_step([])

def _new_game(self) -> None:
    self._wait_for_continue()
    self._run_steps(_boot_steps(self.config.step_ratio))
    self._wait_for_character_select()
    self._select_characters()
    self.expected_health = {"P1": 0, "P2": 0}
    self.expected_wins = {"P1": 0, "P2": 0}
    self._data = self._wait_for_fight_start()

def _sub_step(self, actions: list[Any]) -> dict[str, Any]:
    data = self.emu.step(actions)

    p1_diff = self.expected_health["P1"] - int(data["healthP1"])
    p2_diff = self.expected_health["P2"] - int(data["healthP2"])
    self.expected_health = {
        "P1": int(data["healthP1"]),
        "P2": int(data["healthP2"]),
    }

    data["stunBarP1"] = int(data["stunBarP1"]) >> 24
    data["stunBarP2"] = int(data["stunBarP2"]) >> 24
    data["reward"] = float(p2_diff - p1_diff)
    return data

```

A.4.3 Prompt Construction and Move Restriction

Listing A.3: Prompt construction for text-only and multimodal participants from `src/utils.py`.

```

def create_messages(
    game_info: GameInfo,
    player1: PlayerState,
    player2: PlayerState,
    prev_game_info=None,

```

```

prev_player1=None,
prev_player2=None,
recent_moves=None,
difficulty: str = "expert",
frames: list[str] | None = None,
) -> tuple[list[dict], list[str]]:
    past_info_available = (
        prev_game_info is not None
        and prev_player1 is not None
        and prev_player2 is not None
        and recent_moves is not None
    )

    # game info
    context_prompt = f"Your character: {player2.character}, opponent character: {player1.character}, best of 3:
    you've won {player2.wins} rounds, opponent has won {player1.wins} rounds"

    # moves
    available_moves = get_available_instructions_for_character(
        player2.character, player2.super_art, player2.super_count, difficulty
    )
    if past_info_available:
        # encourage close-in moves to avoid spamming + distancing
        filtered_recent_moves = list(set(recent_moves) - set(CLOSE_IN_MOVES.keys()))
        available_moves = [m for m in available_moves if m not in filtered_recent_moves]
        if not available_moves:
            available_moves = list(CLOSE_IN_MOVES.keys())
    moves_prompt = "You may only use the following moves:\n"
    moves_prompt += chr(10).join("- " + move for move in available_moves)

    if frames:
        messages = [
            {
                "role": "system",
                "content": dedent(
                    f"""
                    You are the most aggressive Street Fighter III 3rd strike player in the world.

                    {context_prompt}
                    {moves_prompt}

                    Simply respond with ONLY the EXACT name of the best move without any surrounding formatting
                    or additional text.
                    """
                ),
            },
            {
                "role": "user",
                "content": [
                    *[
                        {
                            "type": "image",
                            "image": frame,
                        }
                        for frame in frames
                    ],
                    {
                        "type": "text",
                        "text": "Your next move is:",
                    },
                ],
            },
        ]
    else:
        # timer info

```

```

timer_prompt = f"Timer: {game_info.timer}"

# position
p1_box, p2_box = assign_boxes(
    player1.character,
    player1.side,
    player2.character,
    game_info.bboxes,
    game_info.class_ids,
)
distance = None
position_prompt = ""
if p1_box is not None and p2_box is not None:
    p2_x_center = (p2_box[0] + p2_box[2]) / 2
    p1_x_center = (p1_box[0] + p1_box[2]) / 2
    dx = abs(p1_x_center - p2_x_center)
    distance = dx / float(X_SIZE)
if distance is not None:
    if distance > 0.1:
        position_prompt = "You are far away from your opponent. Move closer."
    else:
        position_prompt = "You are close to your opponent. Attack!"
if past_info_available:
    prev_p1_box, prev_p2_box = assign_boxes(
        prev_player1.character,
        prev_player1.side,
        prev_player2.character,
        prev_game_info.bboxes,
        prev_game_info.class_ids,
    )

    if (
        prev_p2_box is not None
        and prev_p1_box is not None
        and p2_box is not None
        and p1_box is not None
    ):
        p2_prev_x = (prev_p2_box[0] + prev_p2_box[2]) / 2
        p2_curr_x = (p2_box[0] + p2_box[2]) / 2
        p2_movement = p2_curr_x - p2_prev_x

        p1_prev_x = (prev_p1_box[0] + prev_p1_box[2]) / 2
        p1_curr_x = (p1_box[0] + p1_box[2]) / 2
        p1_movement = p1_curr_x - p1_prev_x

        if p2_movement * p1_movement < 0: # opposite directions
            if abs(p2_curr_x - p1_curr_x) < abs(p2_prev_x - p1_prev_x):
                position_prompt += (
                    " You are closing distance. Keep going or attack!"
                )
            else:
                position_prompt += " Distance is increasing. Move closer."

# stun
stun_prompt = f"Your stun bar is at {player2.stun_bar / STUN_BAR_MAX * 100}% . Your opponent's stun bar
is at {player1.stun_bar / STUN_BAR_MAX * 100}%."
if player2.stunned:
    stun_prompt += " You are stunned."
if player1.stunned:
    stun_prompt += " Your opponent is stunned."
if past_info_available:
    p2_stun_change = player2.stun_bar - prev_player2.stun_bar
    p1_stun_change = player1.stun_bar - prev_player1.stun_bar
    if p2_stun_change > 0:
        stun_prompt += f" Your stun bar increased by {p2_stun_change / STUN_BAR_MAX * 100}%."

```

```

        if p1_stun_change > 0:
            stun_prompt += f" You inflicted {p1_stun_change / STUN_BAR_MAX * 100}% stun on your opponent."

    # health
    health_prompt = f"Your health is at {player2.health / HEALTH_MAX * 100}%. Your opponent's health is at {player1.health / HEALTH_MAX * 100}%."
    if past_info_available:
        p2_damage_taken = prev_player2.health - player2.health
        p1_damage_taken = prev_player1.health - player1.health
        if p2_damage_taken > 0:
            health_prompt += (
                f" You've taken {p2_damage_taken / HEALTH_MAX * 100}% damage."
            )
        if p1_damage_taken > 0:
            health_prompt += f" You inflicted {p1_damage_taken / HEALTH_MAX * 100}% damage on your opponent
    ."

    # super bar
    power_prompt = f"Your super bar is at {player2.super_bar / SUPER_BAR_MAX * 100}%. Your opponent's super bar is at {player1.super_bar / SUPER_BAR_MAX * 100}%."
    if past_info_available:
        p2_super_change = player2.super_bar - prev_player2.super_bar
        p1_super_change = player1.super_bar - prev_player1.super_bar
        if p2_super_change > 0:
            power_prompt += f" Your super bar increased by {p2_super_change / SUPER_BAR_MAX * 100}%."
        if p1_super_change > 0:
            power_prompt += f" Opponent's super bar increased by {p1_super_change / SUPER_BAR_MAX * 100}%."

    messages = [
        {
            "role": "system",
            "content": dedent(
                f"""
                You are the most aggressive Street Fighter III 3rd strike player in the world.

                {context_prompt}
                {timer_prompt}
                {position_prompt}
                {health_prompt}
                {stun_prompt}
                {power_prompt}
                {moves_prompt}

                Simply respond with ONLY the EXACT name of the best move without any surrounding formatting
                or additional text.
                """
            ),
        },
        {
            "role": "user",
            "content": "Your next move is:",
        },
    ]

    return messages, available_moves

```

A.4.4 YOLO-Based Character Detection

Listing A.4: YOLO service boot and character-detection path from `src/serve/yolo.py`.

```

class YOLOServer:
    @modal.enter()
    async def enter(self):

```



```

import cv2
import numpy as np
import onnxruntime

onnxruntime.set_seed(seed)
onnxruntime.preload_dlls()

await cache_volume.reload.aio()
trt_cache_path.mkdir(parents=True, exist_ok=True)
print(f>Loading model from {model_name}")

self.session = onnxruntime.InferenceSession(
    model_name,
    providers=[
        (
            "TensorrtExecutionProvider",
            {
                "trt_engine_cache_enable": True,
                "trt_engine_cache_path": str(trt_cache_path),
            },
        ),
        "CUDAExecutionProvider",
    ],
)

model_inputs = self.session.get_inputs()
self.input_names = [model_inputs[i].name for i in range(len(model_inputs))]

self.input_shape = model_inputs[0].shape
self.input_height = self.input_shape[2]
self.input_width = self.input_shape[3]

model_outputs = self.session.get_outputs()
self.output_names = [model_outputs[i].name for i in range(len(model_outputs))]

# warm up model

frame = np.random.randint(0, 256, (Y_SIZE, X_SIZE, 3), dtype=np.uint8)

self.img_height, self.img_width = frame.shape[:2]
input_img = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
input_img = cv2.resize(input_img, (self.input_width, self.input_height))

input_img = input_img / 255.0
input_img = input_img.transpose(2, 0, 1)
input_tensor = input_img[np.newaxis, :, :, :].astype(np.float16)

_ = self.session.run(
    self.output_names,
    {self.input_names[0]: input_tensor},
)

@modal.method()
async def boot(self): # so don't have to call 'detect_characters' to boot
    pass

@modal.method()
async def detect_characters(
    self,
    character_ids: list[int],
    frame=None,
    confidence_threshold: float = 0.0,
    return_objects: bool = True,
):
    import cv2

```

```

import numpy as np

if frame is None:
    frame = cv2.imread("/root/example.png")

self.img_height, self.img_width = frame.shape[:2]
input_img = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
input_img = cv2.resize(input_img, (self.input_width, self.input_height))

input_img = input_img / 255.0
input_img = input_img.transpose(2, 0, 1)
input_tensor = input_img[np.newaxis, :, :, :].astype(np.float16)

# run inference

outputs = self.session.run(
    self.output_names,
    {self.input_names[0]: input_tensor},
)

predictions = np.squeeze(outputs[0])
boxes, filtered_class_ids = postprocess_character_detections(
    predictions=predictions,
    character_ids=character_ids,
    confidence_threshold=confidence_threshold,
    input_width=self.input_width,
    input_height=self.input_height,
    img_width=self.img_width,
    img_height=self.img_height,
)
print(
    f"Detected {sum(class_id != -1 for class_id in filtered_class_ids)} characters"
)

if return_objects:
    return boxes, filtered_class_ids

```

A.4.5 Constrained Decoding for a Text-Only Model Server

Listing A.5: Qwen SGLang server with move-constrained decoding from `src/serve/qwen35_35ba3b_fp8.py`.

```

class Qwen35Server:
    ckpt_path: str = modal.parameter(default="")

    @modal.enter()
    async def enter(self):
        import sglang as sgl

        load_path = self.ckpt_path or model_name
        revision = None if self.ckpt_path else model_revision
        print(f>Loading model from {load_path}")

        self.llm = sgl.Engine(
            model_path=str(load_path),
            revision=revision,
            context_length=2048,
            chunked_prefill_size=8192,
            max_running_requests=max_num_seqs,
            cuda_graph_max_bs=max_inputs * 2,
            mem_fraction_static=0.9,
            kv_cache_dtype="fp8_e4m3",
            grammar_backend="xgrammar",
            speculative_algorithm="NEXTN",

```

```

        speculative_num_steps=3,
        speculative_eagle_topk=1,
        speculative_num_draft_tokens=4,
    )
    self.tokenizer = self.llm.tokenizer_manager.tokenizer

    self.sampling_params = {
        "temperature": 0.7,
        "top_p": 0.8,
        "top_k": 20,
        "min_p": 0.0,
        "presence_penalty": 1.5,
        "repetition_penalty": 1.0,
        "max_new_tokens": 32, # roughly len("<longest meta instruction name>")
    }

    # warm up model

    messages, _, _, _, _ = create_random_messages()

    _ = await self.llm.async_generate(
        [
            self.tokenizer.apply_chat_template(
                messages,
                tokenize=False,
                add_generation_prompt=True,
                return_dict=False,
                enable_thinking=False,
            )
        ],
        sampling_params=self.sampling_params,
    )

    @modal.method()
    async def boot(self): # so don't have to call 'chat' to boot
        pass

    @modal.method()
    async def chat(
        self,
        messages: list[dict[str, str]],
        character: str,
        super_art: int,
        super_count: int,
        side: int,
        available_moves: list[str] | None = None,
    ) -> tuple[list[int], str]:
        if available_moves is None:
            available_moves = get_available_instructions_for_character(
                character, super_art, super_count
            )

        sampling_params = {
            **self.sampling_params,
            "regex": "(?:"
            + "|".join(
                sorted(
                    (re.escape(move).replace(r"\ ", " ") for move in available_moves),
                    key=len,
                    reverse=True,
                )
            )
            + ")",
        }
        outputs = await self.llm.async_generate(

```

```

        [
            self.tokenizer.apply_chat_template(
                messages,
                tokenize=False,
                add_generation_prompt=True,
                return_dict=False,
                enable_thinking=False,
            )
        ],
        sampling_params=sampling_params,
    )
    move_name = outputs[0]["text"].strip()

    move_sequence, resolved_move_name = resolve_move_with_fallback(
        character, move_name, side
    )
    if resolved_move_name == "No-Move":
        print(f"Invalid move: {move_name}")
    return move_sequence, resolved_move_name

```

A.4.6 Automated Evaluation Episode Loop

Listing A.6: Shared evaluation episode loop from `src/evaluate.py`.

```

async def run_episode(
    env: Any,
    p1_key: str,
    p1_llm: Any,
    p2_key: str,
    p2_llm: Any,
    yolo: Any | None,
) -> str | None:
    try:
        observation, _ = await asyncio.wait_for(
            asyncio.to_thread(env.reset),
            timeout=env.create_timeout,
        )
    except (OSError, TimeoutError, asyncio.TimeoutError) as exc:
        raise EnvRuntimeError(f"env reset failed: {type(exc).__name__}: {exc}") from exc
    episode_started = time.monotonic()

    prev_game_info = None
    prev_player1_state = None
    prev_player2_state = None
    p1_recent_moves = deque(maxlen=recent_move_limit)
    p2_recent_moves = deque(maxlen=recent_move_limit)
    steps_without_reward = 0
    decision_turns = 0
    finished_normally = False

    while True:
        elapsed = time.monotonic() - episode_started
        if elapsed >= max_episode_runtime_secs:
            warnings.warn(f"terminating eval early: episode runtime hit {elapsed:.1f}s")
            break

        frame = observation["frame"]
        yolo_task = None
        if yolo is not None:
            yolo_task = asyncio.create_task(
                yolo.detect_characters.remote aio(
                    [CHARACTER_TO_ID[character], CHARACTER_TO_ID[character]],
                    frame,

```

```

    )
)

frame_data = None
if MODEL_SPECS[p1_key]["uses_frames"] or MODEL_SPECS[p2_key]["uses_frames"]:
    from PIL import Image

    buffer = BytesIO()
    Image.fromarray(frame).save(buffer, format="JPEG", quality=85)
    frame_data = "data:image/jpeg;base64," + base64.b64encode(
        buffer.getvalue()
    ).decode("utf-8")

obs_p1 = observation["P1"]
obs_p2 = observation["P2"]
if yolo_task is not None:
    boxes, class_ids = await yolo_task
else:
    boxes, class_ids = [], []
game_info = GameInfo(
    timer=int(observation["timer"][0]),
    boxes=boxes,
    class_ids=class_ids,
)
p1_side = int(obs_p1["side"])

player1 = PlayerState(
    character=character,
    super_art=super_art,
    wins=int(obs_p1["wins"][0]),
    side=p1_side,
    stunned=bool(obs_p1["stunned"]),
    stun_bar=int(obs_p1["stun_bar"][0]),
    health=int(obs_p1["health"][0]),
    super_count=int(obs_p1["super_count"][0]),
    super_bar=int(obs_p1["super_bar"][0]),
)
player2 = PlayerState(
    character=character,
    super_art=super_art,
    wins=int(obs_p2["wins"][0]),
    side=int(obs_p2["side"]),
    stunned=bool(obs_p2["stunned"]),
    stun_bar=int(obs_p2["stun_bar"][0]),
    health=int(obs_p2["health"][0]),
    super_count=int(obs_p2["super_count"][0]),
    super_bar=int(obs_p2["super_bar"][0]),
)

p1_messages, p1_available_moves = create_messages(
    game_info,
    player2,
    player1,
    prev_game_info,
    prev_player2_state,
    prev_player1_state,
    p1_recent_moves,
    frames=[frame_data] if MODEL_SPECS[p1_key]["uses_frames"] else None,
)
p2_messages, p2_available_moves = create_messages(
    game_info,
    player1,
    player2,
    prev_game_info,
    prev_player1_state,

```

```

    prev_player2_state,
    p2_recent_moves,
    frames=[frame_data] if MODEL_SPECS[p2_key]["uses_frames"] else None,
)

p1_result, p2_result = await asyncio.gather(
    p1_llm.chat.remote.aio(
        p1_messages,
        character,
        super_art,
        int(obs_p1["super_count"][0]),
        p1_side,
        p1_available_moves,
    ),
    p2_llm.chat.remote.aio(
        p2_messages,
        character,
        super_art,
        int(obs_p2["super_count"][0]),
        int(obs_p2["side"]),
        p2_available_moves,
    ),
)

p1_buttons, p1_move_name = p1_result
p2_buttons, p2_move_name = p2_result
decision_turns += 1

if len(p1_buttons) > len(p2_buttons):
    p2_buttons = p2_buttons + [0] * (len(p1_buttons) - len(p2_buttons))
elif len(p2_buttons) > len(p1_buttons):
    p1_buttons = p1_buttons + [0] * (len(p2_buttons) - len(p1_buttons))

total_reward = 0
terminated = False
truncated = False
for p1_button, p2_button in zip(p1_buttons, p2_buttons):
    try:
        observation, reward, terminated, truncated, _ = await asyncio.wait_for(
            asyncio.to_thread(
                env.step,
                {
                    "agent_0": p1_button,
                    "agent_1": p2_button,
                },
            ),
            timeout=env_step_timeout,
        )
    except (OSError, TimeoutError, asyncio.TimeoutError) as exc:
        raise EnvRuntimeError(
            f"env step failed: {type(exc).__name__}: {exc}"
        ) from exc
    total_reward += reward

p1_recent_moves.append(p1_move_name)
p2_recent_moves.append(p2_move_name)
prev_game_info = game_info
prev_player1_state = player1
prev_player2_state = player2

if total_reward == 0:
    steps_without_reward += 1
    if steps_without_reward >= max_steps_without_reward:
        warnings.warn(
            f"terminating eval early: {steps_without_reward} steps without reward"

```

```

        )
        break
    else:
        steps_without_reward = 0

    if decision_turns >= max_decision_turns:
        warnings.warn(
            f"terminating eval early: {decision_turns} decision turns reached"
        )
        break

    if terminated or truncated:
        finished_normally = True
        break

if not finished_normally:
    return None

p1_wins = int(observation["P1"]["wins"][0])
p2_wins = int(observation["P2"]["wins"][0])
if p1_wins > p2_wins:
    return p1_key
if p2_wins > p1_wins:
    return p2_key
return None

```

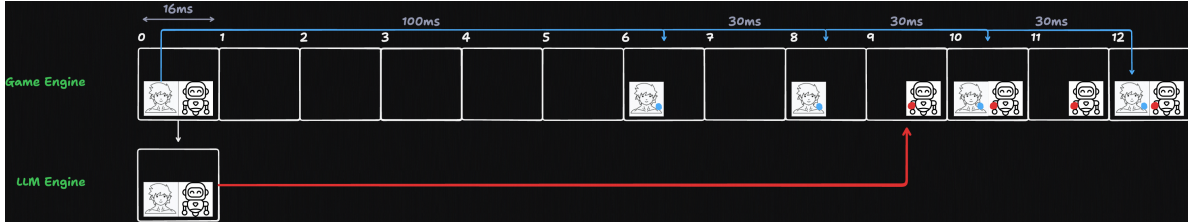


Figure A.1: Illustrative latency budget for LLM inference.

A.5 Latency and Throughput Calculations

A system running at 60 frames per second emits one frame roughly every

$$\frac{1000 \text{ ms}}{60} \approx 16.7 \text{ ms.}$$

If a typical action contains four button presses, and a human requires about 100 ms for the initial reaction plus about 30 ms for each of the next three button presses, then a rough estimate of a human speed for one action is

$$100 + 3 \times 30 = 190 \text{ ms/action.}$$

If the system plays a chosen action immediately and then emits the remaining three button presses on subsequent frames, those three follow-up inputs consume about

$$3 \times 16 \approx 48 \text{ ms.}$$

That leaves an approximate model-response budget of

$$190 - 3 \times 16 \approx 142 \text{ ms/action.}$$